TLAMATINI

Complete Project Dossier: what the system does, how it works, how to use Tlamatini, complete repository file tree, and effective line inventory

Measure	Value
Generated	June 19, 2026 12:03 PM UTC-0600
Current HEAD	d6a11b3 - Fixing the ollama token stuff in docs.
Resolved version	1.26.0 (git)
Repository inventory files	740
Tracked files	740
Git-unignored working-tree additions	0
Workflow agents	82
Multi-Turn tools	89
Skills	27
Total effective lines	152,952
Total physical text lines	210,000

1. What Tlamatini Is

- Tlamatini is a self-hosted AI developer assistant (cloud LLMs by default; the app and RAG run locally) built with Django, Django Channels, LangChain, LangGraph, FAISS/BM25 retrieval, and a large in-repository agent application.
- She combines a browser chat surface, a Retrieval-Augmented Generation stack, a Multi-Turn tool executor, MCP-backed context providers, wrapped chat-agent runtimes, and a visual Agentic Control Panel for workflow design.
- She is designed for development operations: codebase analysis, file and directory context, deterministic file discovery/search/editing, command execution, Python execution, screenshots, web/search helpers, notifications and attention routing, DevOps tools, local model operation, Windows packaging and uninstall registration, first-person self-knowledge about her own runtime, and embedded-firmware control for STM32F4, ESP32-class, Arduino-class, and ESPHome smart-home boards.

What the system does

- Answers codebase questions with loaded file or directory context.
- Uses hybrid retrieval to extract metadata, split content, rank source chunks, and respect context budgets.
- Can discover files by glob pattern, search their contents by regex, and make surgical in-place replacements through the Globber, Grepper, and Editor agent/tool trio.
- Can connect to external MCP servers declared in `external_mcps.json`, expose their remote tools to Multi-Turn under the `ext__<server>__<tool>` naming convention, and supervise those connections through status / reconnect / doctor / import / list / call helpers.
- Gives operators GUI-first database maintenance through the new DB dropdown for backup and staged database replacement.
- Lets operators manage provider secrets from the browser through the Config -> Access Keys Wizard instead of hand-editing `config.json`.
- Can drive Blender through the Blenderer agent, using the official Blender MCP add-on socket to inspect scenes, mutate objects and materials, run raw code, and automate renders from chat or the workflow canvas.
- Can pause before every state-changing Multi-Turn execution and ask the operator to approve or deny that exact step through the Ask Execs checkbox.
- Can raise a Windows attention signal when the browser needs the operator: Ask Execs prompts and Notifier events can flash Tlamatini's own taskbar presence and leave an uppercase banner in `tlamatini.log`.
- Registers packaged installs in Windows `Installed apps` / `Programs and Features` with a real uninstall entry, so the release behaves like a normal installed application instead of only a shortcut bundle.
- Can update packaged installs in place through About -> Check for updates: she checks the latest GitHub release, stages the download, swaps locked files externally, and preserves operator state such as `config.json`, the database, and content.
- Hardens generated files: File-Creator's bulk-write path now preserves long content byte-complete even when it contains heavy quoting or semicolon-rich source text.
- Warns GPU-host operators before a directory-context load is likely to saturate VRAM and degrade embedding throughput.
- Exposes a coherent versioning surface across builds, runtime UI, logs, and an open health-check endpoint.
- Carries a first-person self-knowledge map so she can answer more accurately about her own architecture, ports, runtime modes, pages, and capabilities.
- Can command Kali Linux offensive-security tooling through MCP-Kali-Server for authorized recon, enumeration, web scanning, and assessment workflows.
- Can diagnose an external MCP before the first live connection through the MCP Doctor agent and wrapped `chat_agent_mcp_doctor` tool, checking transport, runtime requirements, PATH availability, placeholder secrets, and the next operator step.
- Can scaffold, author, build, flash, reset, and observe STM32F4 firmware through STM32er and the STM32 Template Project MCP, with a fail-safe preflight before any hardware mutation.

- Can scaffold, author, build, upload, and monitor ESP32-class firmware through ESP32er and PlatformIO Core, with zero-config bootstrap and a serial-aware preflight before hardware mutation.
- Can author YAML-based smart-home firmware through ESPHomer and ESPHome, including zero-config bootstrap, device-config generation, validation, compile, USB/OTA upload, and bounded log observation for ESP32 / ESP8266 / RP2040 / BK72xx devices.
- Can play media on the operator's machine: an audio file to the speakers through AudioPlayer (soundfile + sounddevice — volume in percent and a time-played budget that truncates a longer file or loops a shorter one), or a video file with audio on a chosen display through VideoPlayer (ffpyplayer, whose wheel bundles ffmpeg + SDL, plus an OpenCV window — display, volume, the same truncate/loop time budget, window size, and fullscreen); both are observational/output and ship on the canvas and as wrapped chat tools.
- Can SPEAK and LISTEN: Talker (text-to-speech) renders input_text to a 24 kHz WAV through an Ollama neural TTS model (default Orpheus-3b-FT) and is female-voice-only by design, while Whisperer (speech-to-text) records the microphone itself or transcribes a file via faster-whisper locally (NVIDIA-GPU auto-detect with an always-present CPU fallback) or a cloud Whisper API; both light a zero-latency console REC indicator driven by the live audio stream and are observational/output, on the canvas and as wrapped chat tools.
- Can manage real desktop windows by title: focus them, tile them, resize them, list them, and close them deterministically through Win32 calls.
- Can drive a real Playwright browser through scripted interactive steps for logins, forms, assertions, downloads, extraction, and end-to-end UI checks.
- Can drive a live Unreal Engine 5 editor through the Unreal MCP plugin, from either Multi-Turn chat or the visual workflow canvas.
- Actively reaps orphaned Windows console-host and pool-child processes so long Multi-Turn or ACPX sessions do not leave misleading Tlamatini-icon ghosts in Task Manager.
- Can autonomously kill only genuinely hung shell-wrapper subtrees through a boot-time command watchdog that measures CPU and I/O progress instead of guessing from elapsed time.
- Runs checked Multi-Turn requests through request-scoped planning, capability selection, tool calls, observations, monitoring, and final synthesis.
- Can optionally inspect and modify her own bundled source tree in self-modify builds after verifying that `TlamatiniSourceCode/` is actually present.
- Launches wrapped copies of selected workflow agents in isolated runtime folders without mutating templates.
- Lets users design, validate, save, pause, resume, and stop visual workflows through the Agentic Control Panel.
- Turns successful Multi-Turn tool executions into starter `.flw` workflows that can be inspected and validated in ACP.
- Packages the project into a distributable Windows release with installer and uninstaller tooling.

How it works

- Browser UI sends chat and workflow requests through Django views and Channels WebSockets.
- RAG chains load selected file/directory context, retrieve relevant chunks, and build answer prompts.
- DB-menu actions validate directories or SQLite files in the browser, then call Django views that either copy the live database out or stage a replacement into `DB/ToLoad/db.sqlite3`.
- Config -> Access Keys Wizard reads masked provider-key status from the backend and persists only the edited secrets, keeping the browser flow honest without dumping live values back to the page.
- The file-navigation and file-edit trio sits above raw shell execution: Globber enumerates matching files, Grepper scans content with regex while pruning noisy/binary trees, and Editor performs byte-exact in-place replacements without rewriting an entire file.
- External MCP connectivity is catalog-driven: `external_mcps.json` stores Claude-style `mcpServers` entries, `external_mcp_manager.py` negotiates stdio / streamable-HTTP / SSE / WebSocket transports, and wrapped remote tools are surfaced into the planner as `ext__<server>__<tool>` only after the connection is healthy.
- The MCP Doctor path is intentionally safer than a live connect: it reads the configured server entry, validates transport shape, runtime commands, PATH/toolchain presence, placeholder secrets, and docs/source URLs, then returns an onboarding diagnosis without consuming the server's real tool surface.

- Blenderer opens the official Blender MCP add-on TCP socket (default `localhost:9876`), sends one action payload or raw code-execution request, and returns the structured result through the same wrapped-tool / canvas contract used by the rest of the agent catalog.
- When Ask Execs is enabled, the synchronous Multi-Turn executor stops before each state-changing tool call, emits an `exec_permission_request`, and waits on `ExecPermissionBroker` until the browser sends Proceed or Deny.
- When the browser surfaces an Ask Execs prompt or a Notifier event, JavaScript can POST to `/agent/flash_window/`; the backend then best-effort flashes the `Tlamatini.exe` console/taskbar window through `window_flash.py` and prints an uppercase attention banner for the log.
- Installer-time registration writes a per-user HKCU Add/Remove Programs entry pointing at `Uninstaller.exe`, and frozen startup re-checks that entry through `windows_app_registration.self_heal_for_frozen()` so older installs retroactively appear in Windows' uninstall surfaces.
- In-app self-update uses Django views plus `self_update.py` to check GitHub releases, download and stage the selected package, then hand off the locked-file swap to the external `apply_update.ps1` helper before relaunch.
- Frozen-build hardening now verifies media dependencies during packaging too: `build.py` embeds numpy and OpenCV in both shipped Python runtimes and refuses to produce a release if those imports are missing.
- Before a heavy directory embedding run on supported NVIDIA hosts, a fail-open pre-flight guard can estimate VRAM pressure and surface a non-blocking warning in chat.
- Version resolution now flows through git tags, a runtime resolver module, generated build artefacts, and an open `/agent/version/` endpoint.
- A first-person self-knowledge file (`Tlamatini.md`) is injected into prompt construction for all chains, but loaded user context still outranks that self-reference when the request is a generic summary of the provided project.
- When Multi-Turn is enabled, the global planner selects context and tool stages before the executor binds only the relevant tools, including wrapped deterministic agents such as De-Compresser.
- Optional self-modify builds bundle `TlamatiniSourceCode/`; `copy_source_assets.py` generates that snapshot with rebuild instructions and redacted secrets, and prompt rules require her to verify that directory exists before claiming she can inspect or change her own code.
- The Kalier path talks directly to the MCP-Kali-Server Flask API over HTTP with Python-stdlib `urllib`, auto-seeding the default box from `kali_server_url` in Config -> URLs before any one-off per-call override is applied, and captures one atomic `INI_SECTION_KALIER` block per run.
- The STM32er path spawns the STM32 Template Project MCP stdio server, performs the MCP initialize handshake, runs exactly one requested tool or composite action, and emits one atomic `INI_SECTION_STM32ER` block with the result, project directory, and stage metadata.
- Before any flash-capable STM32er action, a critical-mission preflight validates the arm-none-eabi toolchain, STM32CubeIDE, programmer path, ST-LINK presence, and STM32F-family match; compile-only steps can run boardless, but unsafe hardware mutations are refused fail-safe.
- The ESP32er path resolves or bootstraps PlatformIO Core, invokes `pio` subcommands directly with Python-stdlib process control, validates project and serial-port readiness, and emits one atomic `INI_SECTION_ESP32ER` block with stage, project, port, and stdout/stderr payloads.
- The ESPHomer path resolves or bootstraps the `esphome` CLI, can generate a minimal valid YAML device config headlessly, validates either a serial board or OTA host before upload/log actions, and emits one atomic `INI_SECTION_ESPHOMER` block with action, config path, stage, and captured CLI output.
- The Windower path uses Win32 APIs plus the cross-process `AttachThreadInput` focus-transfer dance to locate windows by title and apply one lifecycle action while still returning structured geometry/state fields.
- The Playwrighter path loads a declarative step list, drives Playwright against Chromium/Firefox/WebKit, and emits one atomic `INI_SECTION_PLAYWRIGHTER` block with status, assertions, extracted values, and the final URL.
- The Unrealer path opens a TCP socket to the Unreal MCP plugin, sends one `{"type": command, "params": {...}}` payload, captures the JSON reply, and emits one `INI_SECTION_UNREALER` block for downstream logic.
- A boot-time command watchdog samples CPU time and I/O bytes across shell-interpreter subtrees and reaps only the ones that stay idle past the grace-and-streak window, protecting healthy long-running commands while rescuing wedged prompt waits.

- After spawn-capable tool calls and again after the final answer, the orphan reaper can sweep dead descendants, orphaned `conhost.exe` companions, and stale pool-linked processes without ever raising into the chat path.
- Tool calls execute in the backend, append observations, and may create wrapped runtime copies under `agent/agents/pools/_chat_runs_/`.
- On the next full start-up, `manage.py` can swap a staged database into place before Django imports, while archiving the previous live database under `DB/Older/<timestamp>/`.
- ACP flows deploy session-scoped pool instances, wire config values, validate NxN graph rules, and execute through Starter-driven flow semantics.
- Build scripts collect static assets, bundle Django/Python resources, add agent templates, and assemble `pkg.zip`, `Uninstaller.exe`, and `dist/Tlamatini_Release/`; `build.py --self-modify` additionally injects the generated source snapshot for self-rebuildable releases.

2. Architecture Layers

Layer	Role
Browser interfaces	Chat page plus Agentic Control Panel templates and JavaScript modules.
Django/Channels	Authentication, views, WebSockets, session state, message persistence, and ASGI startup.
RAG and context	Metadata extraction, text splitting, FAISS/BM25 retrieval, context budgeting, and fallback behavior.
Multi-Turn engine	Capability registry, global execution planner, explicit tool loop, answer parsing, and answer-success classification.
Tools and agents	Core tools, MCP context providers, wrapped chat-agent launchers, and the current visual workflow agent templates.
Packaging	PyInstaller build scripts, shortcut registration, `.flw` association, installer, uninstaller, and release folder assembly.

Design principles

- Evidence-first answers: Tlamatini grounds responses in selected project context and hybrid retrieval rather than freeform model memory.
- Explicit orchestration: checked Multi-Turn uses a visible tool loop, capability scoring, and staged planning instead of a single opaque call.
- Operational reversibility: risky changes such as database replacement are staged, archived, and delayed to the only safe window instead of hot-swapped mid-session.
- Fail-open diagnostics: GPU pressure probes and session-restore safeguards warn early without breaking CPU-only or degraded environments.
- Runtime isolation: wrapped chat-agent copies run in session-scoped folders so template agents remain pristine while live runs stay inspectable.
- Self-knowledge with scope discipline: she can talk accurately about herself without letting self-reference override a user-loaded project context.
- Operator truth over vibes: Exec Report tables, tlamatini.log, skill audits, and ACPX transcripts make the system auditable after execution.

3. Installation, Configuration, and Everyday Use

Start here - the easiest path

- BookOfTlamatini now leads with a five-step onboarding path because the easiest way to succeed with Tlamatini is to treat setup as one guided sequence instead of reading the whole handbook first.
- Recommended path for most operators: install the packaged release from GitHub Releases, launch the Start-menu shortcut, and let the bundled Python 3.12.10 plus dependencies carry the runtime without asking the user to install Python manually.
- Developer path stays available: clone the repo, create a virtual environment, install `requirements.txt`, run migrations, and start Django with `python Tlamatini/manage.py runserver --noreload`.
- After the app opens, the first in-app surfaces that matter are `Config -> Models`, `Config -> URLs`, and `Config -> Access Keys Wizard`; those now form the real beginner path, not manual JSON editing.
- For ordinary question-answering keep Multi-Turn off; turn it on only when you want Tlamatini to execute tools, wrapped agents, or remote MCP capabilities instead of answering directly.

Installation essentials

- The easiest path for most users is the packaged installer from GitHub Releases: no manual Python install is required because the release already carries Python 3.12.10 and the project dependencies.
- Source mode remains the developer path: clone the repo, create a virtual environment, install `requirements.txt`, run migrations, create a superuser, collect static files, and then launch Django.
- Packaged Windows installs open the browser at `http://127.0.0.1:8000/` and create the default `user / changeme` login; manual source installs use your own `createsuperuser` account instead.
- When a newer packaged release exists, the intended upgrade path is in-app: `About -> Check for updates`, not manual folder replacement.
- You can run either the checked-in cloud/back-end defaults from `Tlamatini/agent/config.json` or a local/remote Ollama-backed configuration with matching model names.

First configuration screens

- Three new onboarding screenshots now anchor the first-run path: `Tlamatini/agent/images/MenuConfig.jpg`, `ConfigureModels.jpg`, and `ACPXKeysConfigureWizard.jpg`.
- `Config -> Models` is the place where operators map embedding, chat, vision, and auxiliary model names to what Ollama actually exposes on the host machine.
- `Config -> Access Keys Wizard` now carries an especially important rule: a localhost Ollama usually needs no Ollama token, while a remote Ollama endpoint may require one.
- Saved model, URL, or credential changes can invalidate the assumptions of the current session, so reconnecting after major Config edits is now part of the honest operator guidance.

Configuration essentials

- Source mode resolves `Tlamatini/agent/config.json`; frozen builds resolve `config.json` next to the executable; `CONFIG_PATH` overrides both.
- Core keys include `embeding-model`, `chained-model`, `ollama_base_url`, `ollama_token`, `enable_unified_agent`, `unified_agent_model`, and `unified_agent_max_iterations`.
- The checked-in default model baseline moved again in the recent Git window: the shared config now favors `kimi-k2.6:cloud`, so source or frozen installs that keep the shipped config should be documented as cloud-first unless the operator intentionally swaps models.
- URL configuration now also includes `kali_server_url`, the STM32er bootstrap fields `stm32_mcp_server_script`, `stm32_mcp_python`, `stm32_template_dir`, `stm32_ide_root`, `stm32_mcp_repo_url`, and `stm32_mcp_install_dir`, plus ESP32er's `pio_executable` and `pio_core_dir`, all edited from `Config -> URLs` and inherited automatically by the chat-side wrapped tools.

- Credential configuration is no longer hand-edit-only: Config -> Access Keys Wizard provides a browser-side path for ACPX and provider secrets while preserving masked status in the UI.
- The chat-side Config -> Models and Config -> URLs dialogs are now first-class configuration surfaces, and they can explicitly ask the operator to reconnect when saved values change live-session assumptions.
- The separate DB dropdown is not a config editor: it is a maintenance surface for copying the live SQLite database out or staging a replacement for the next full start-up.
- Multi-Turn is toggled from the chat toolbar, but it depends on the unified-agent configuration and the selected model/base-url pairing being valid; the current default iteration ceiling is 4096, and Ask Execs only becomes available when Multi-Turn itself is on.
- Image interpretation can run through Claude-backed cloud paths or Qwen/Ollama-backed local paths, and remote Ollama can be protected with a bearer token.

DB menu and database swap-in

- The new DB dropdown gives operators two GUI-first maintenance paths: `Backup database` copies the live SQLite file out, and `Set DB` stages a chosen `db.sqlite3` for the next full start-up.
- Both dialogs are live-validated in the browser and now expose Browse buttons that open native host-side pickers for folders or `db.sqlite3` files.
- Backup is read-only and uses the currently live database path; Set DB never hot-swaps the file mid-session because Django already holds SQLite open.
- Set DB writes the selected file to `DB/ToLoad/db.sqlite3`; the real swap happens only at the top of `manage.py` before Django imports anything.
- When that swap runs, the previous live database is moved into `DB/Older/<timestamp>/db.sqlite3`, creating a built-in rollback trail instead of overwriting history.
- Reconnect is not enough for this path: the operator must fully restart Tlamatini so the pre-Django swap window opens again.
- If the staged file is bad or locked, startup fails open: Tlamatini logs the error and continues with the previous live database.

Versioning system

- Tlamatini now follows Semantic Versioning 2.0.0 with git tags as the single source of truth: you tag, then you build, instead of hand-editing version strings across files.
- The build path resolves a version once and propagates it into generated runtime metadata, Win32 VERSIONINFO resources, and the release-folder naming convention.
- On untagged commits the resolver falls back honestly to a git-derived development version instead of failing the build or lying about the release state.

Version surfaces

- Operators can now see the running version in the About dialog, the startup banner, the open `GET /agent/version/` health-check endpoint, and Windows file properties on the built executables.
- Packaged installs also project the same release identity into Windows' Installed-apps metadata through the ARP `DisplayVersion` field, so the uninstall surface and the binaries agree on the version being removed.
- The runtime resolver lives in `Tlamatini/agent/version.py`, while the build-oriented coordination logic lives in the repo-root `versioning.py` and the longer policy notes live in `VERSIONING.md`.
- Build overrides are supported in a predictable order: CLI `--version`, then `TLAMATINI_VERSION`, then git describe, then the sentinel `0.0.0+unknown`.

Current release focus in v1.26.0

- The repository currently resolves to `v1.26.0` from the latest tag `30a87f3`, the External MCPs release; its defining feature is the External MCPs universal client (connect to any MCP server over stdio / streamable-HTTP / SSE / WebSocket, the MCP Doctor diagnostic agent, full-surface Multi-Turn tool binding, and Step-by-Step setup mode), with the earlier

ESPHomer smart-home firmware lane already contained in this version.

- Operators reach that capability from a first-class UI surface: the `External -> MCPs` menu manages the server catalog, imports or exports Claude-style JSON, and can progressively guide setup instead of assuming the user already knows every transport and runtime field.
- Data-preserving update is the key fix: the live database sits inside the PyInstaller `_internal/` folder, which an update replaces wholesale, so a naive swap would wipe chat history and custom toggles. `apply_update.ps1` now stages the user's database through `DB/ToLoad/` and drops a `post_update_migrate.flag`; on the next launch `manage.py` swaps that database back over the freshly shipped one and runs `migrate` in a child process, so the user keeps their history and toggles and still receives new agent / tool / prompt rows.
- The second pillar is media-agent reliability: numpy and OpenCV (`cv2`) are now embedded in both the carried Python that runs the pool agents and the frozen `_internal`, with `build.py` asserting both imports so the build aborts loudly rather than shipping a Recorder, Camcorder, AudioPlayer, VideoPlayer, or Whisperer that would crash at runtime for a missing native library.
- The broader operator surface around this release is larger too: Globber, Grepper, and Editor give Tlamatini deterministic file discovery, regex search, and surgical in-place editing, the firmware stack already includes ESPHomer for ESPHome-based smart-home devices, and v1.26.0 adds the External MCPs universal client so Tlamatini can use the tools of any external MCP server declared in JSON.

External MCP universal client

- The `v1.26.0` headline feature is the External MCP universal client: Tlamatini can now consume any MCP server declared in `external_mcps.json` instead of depending only on bundled MCP integrations.
- Four transports are supported in the current implementation — stdio, streamable HTTP, legacy SSE, and WebSocket — and the active server set is intentionally bounded so operators can expose high-value remote tools without flooding the planner.
- Once connected, remote MCP tools are wrapped into the same Multi-Turn execution surface under `ext__<server>__<tool>`, so they participate in planning, execution reporting, Ask Execs, and the rest of the operator guardrails instead of becoming a parallel hidden subsystem.

MCP Doctor agent and wrapped tool

- MCP Doctor is the new onboarding and triage specialist added with the External MCP release: she inspects a configured server entry and tells the operator what is wrong before a live connection attempt wastes time.
- The diagnostic checks cover transport shape, runtime command availability on PATH, placeholder or missing secrets, endpoint/source/docs links, and the concrete next step the operator should take to make the server connectable.
- Operators can reach the same behavior from both surfaces that matter: the MCP Doctor workflow agent on the canvas and the wrapped `chat_agent_mcp_doctor` tool inside Multi-Turn.

External MCP implementation assets

- The release is backed by real tracked assets rather than markdown-only claims: `agent/external_mcp_manager.py`, `agent/external_mcps.json`, the `agent/agents/mcp_doctor/` tree, the `0141`-`0143` migrations, `static/agent/js/external_mcps_dialog.js`, and `static/agent/css/external_mcps_dialog.css` are the core implementation wave.
- The current test surface proves this is not a shallow UI feature: `test_external_mcp_universal.py`, `test_external_mcp_transports.py`, `test_external_mcp_e2e.py`, `test_external_mcp_add_flow.py`, and `test_parametrizer_mcp_doctor.py` exercise the universal-client path and the onboarding diagnosis path.
- Handbook and design evidence now exist alongside the code too: `docs/external_mcp_bulletproof_architecture.md`, the new README tutorial, Book sections, capability/planner wiring, and the dialog assets all moved together in the same release wave.

Blenderer

- Blenderer was introduced in `v1.20.0` as the 77th workflow agent and remains the direct bridge to the official Blender MCP add-on, letting Tlamatini operate a live Blender session from either Multi-Turn chat or the visual workflow canvas.

- The bridge talks over a TCP socket (default `localhost:9876`) and supports both raw `execute_code` requests and higher-level scene/object/material/render actions, so operators can mix deterministic verbs with precise Python-driven 3D automation.
- This extends Tlamatini beyond code and firmware orchestration into DCC / 3D-production work: asset inspection, scene mutation, material changes, camera setup, and render-triggering now live inside the same operator surface as the rest of the system.

In-app self-update

- The packaged application now includes an in-app self-update flow exposed from About -> Check for updates, so operators can refresh a frozen install without manually unpacking and replacing the entire release folder.
- The backend checks the latest GitHub release, downloads and stages the package, then hands off the locked-file swap to `apply_update.ps1`, which performs the replacement after the running executable exits and relaunches the updated app.
- The update path is state-preserving by design: it keeps `config.json`, user content, and one `agents_backup` generation, and it stages the user's database through `DB/ToLoad/` plus `post_update_migrate.flag` so the next launch restores and migrates that DB instead of wiping it.

Self-modify source snapshot

- Self-modify builds now rely on `copy_source_assets.py` to generate `TlamatiniSourceCode/` as a fresh rebuild-oriented snapshot of the repository.
- That snapshot includes source, build scripts, docs, skills, and small required binaries while deliberately omitting or redacting heavy media and secrets; `_REBUILD_INSTRUCTIONS.md` plus `_SOURCE_SNAPSHOT_MANIFEST.json` explain how to restore the missing payloads.
- Because the snapshot is optional, the runtime contract remains strict: Tlamatini must verify that `TlamatiniSourceCode/` exists before claiming she can inspect, modify, or rebuild herself.

API-Keys Wizard

- Config now includes an Access Keys Wizard / API-Keys Wizard path so operators can manage provider credentials from the browser instead of editing `config.json` directly.
- The backend exposes masked status plus explicit save endpoints, which keeps the operator flow convenient without echoing raw secrets back into the page.
- That feature arrives as a real asset wave, not just a hidden helper: `agent/access_key_wizard.py`, `static/agent/js/access_keys_wizard.js`, `static/agent/css/access_keys_wizard.css`, and the updated `templates/agent/agent_page.html` now form a dedicated setup surface.
- This matters operationally because ACPX agents, cloud LLM providers, and adjacent integrations often fail for simple missing-key reasons; the wizard turns that setup into a first-class UI workflow.

New assets in the current release wave

- Recent assets worth calling out explicitly now span several release waves: the new onboarding screenshots `agent/images/MenuConfig.jpg`, `agent/images/ConfigureModels.jpg`, and `agent/images/ACPXKeysConfigureWizard.jpg`; `agent/external_mcp_manager.py`, `agent/external_mcps.json`, the `agent/agents/mcp_doctor/` tree, `static/agent/js/external_mcps_dialog.js`, `static/agent/css/external_mcps_dialog.css`, and the `0141`-`0143` migrations for the External MCP + MCP Doctor release; `agent/self_update.py` plus `apply_update.ps1` for the data-preserving updater; `build.py` for numpy/OpenCV embedding checks; the `agent/agents/{editor,grepper,globber}/` directories plus their migrations/tests for deterministic file discovery; and the `agent/agents/esphomer/` tree plus migrations/tests/sample YAML for ESPHome-based smart-home firmware control.
- The current `v1.26.0` window also refreshes shipped visual assets: `TlamatiniAbout.png` replaces the old `TlamatiniAbout.jpg`, and `agent/images/TlamatiniAndKyber.mp4` is now part of the repository asset set described by the dossier.
- The same recent window also retains the earlier self-modify/browser-setup asset wave — `copy_source_assets.py`, `agent/access_key_wizard.py`, `static/agent/js/access_keys_wizard.js`, `static/agent/css/access_keys_wizard.css`, and the Blender control surface in `agent/agents/blenderer/`.

- Key operator/runtime files such as `prompt.pmt`, `chat_agent_registry.py`, `tools.py`, `views.py`, `urls.py`, `manage.py`, `file_extractor.py`, and the File-Creator/File-Extractor templates also changed, so the visible features are backed by concrete implementation assets rather than documentation-only promises.
- Because the dossier already includes the full repository inventory (git-tracked files plus git-unignored working-tree additions) and the full line-count inventory, these named assets serve as the human-readable shortlist of what changed most materially in the latest release wave.

File-Creator hardening

- The `v1.19.5` File-Creator hardening pass fixes the wrong-symbol / wrong-escape corruption path that hit backslash-dense Java, JSON, CSS, and regex files during wrapped chat-agent execution.
- Two byte-exact channels now exist: plain `content` is re-extracted verbatim from the raw request, and `content_b64` can carry source or binary bytes through a parser-immune base64 path when exact escaping matters most.
- For operators, the consequence is straightforward: code generation and document generation can now be described as byte-complete file writes rather than best-effort convenience output.
- This belongs in the dossier because File-Creator is one of Tlamatini's central deterministic execution surfaces and one of the safest alternatives to GUI typing or editor-driving automation.

Self-knowledge and identity contract

- Version `1.8.0` gives Tlamatini a first-person self-knowledge map in `Tlamatini/agent/Tlamatini.md`, so she can answer more accurately about her own architecture, runtime modes, open ports, pages, and internal capability surface.
- That self-reference is injected into all prompt chains through `prompt.pmt` and `agent/rag/config.py`, but it fails open if the file is missing or unreadable and it never overrides a user-loaded project when the request is a generic summary of the provided context.
- The language contract matters too: when a pronoun is used for Tlamatini in the documentation or prompt guidance, she is referred to as `she` / `her`, matching the updated handbook identity rules.

Self-modify builds

- Self-modification is a separate capability axis from frozen-vs-source runtime: only builds created with `python build.py --self-modify` bundle `TlamatiniSourceCode/` next to the application.
- When that directory is present, she can inspect and modify her own shipped source tree; when it is absent, she must say so plainly and fall back to her injected self-knowledge plus the surrounding docs.
- The build pipeline now announces that choice explicitly, so operators can tell whether a release is self-modify-capable before asking her to work on herself.

Multi-Turn 4096-turn autonomy

- The unified-agent loop now defaults to 4096 iterations instead of 256, giving long autonomous operator runs much more room before they exhaust the turn budget.
- That expansion is about conversational/tool-loop depth, not about blindly firing more tools at once: the planner's selected-tool cap still keeps the tool surface bounded per request.
- Operationally, the bigger ceiling helps long workflows, while duplicate-call guards and the dedicated `chat_agent_sleeper` tool remain the antidote to accidental busy-polling loops.

Ask Execs in Multi-Turn

- Introduced in `v1.10.0` and still part of the current `v1.26.0` surface, `Ask Execs` is the Multi-Turn-only safety modifier that makes Tlamatini ask before each state-changing Tool, MCP, wrapped agent, or skill-backed execution instead of running it immediately.
- The permission dialog is explicit and auditable: it names the Tool or Agent family, the underlying raw tool name, the full parameters, the program or command to be executed, and the shell or execution surface involved.
- Proceed runs that one step and then prompts again at the next state-changing step; Deny halts the entire chain immediately and appends a red `Execution interrupted` banner even when Exec Report itself is off.

Ask Execs runtime path

- Under the hood, `agent/exec_permission.py` provides `ExecPermissionBroker`, which lets the synchronous worker-thread executor emit a permission request onto the WebSocket event loop and then block on a `threading.Event` until the browser replies.
- The gate sits after deduplication and quota checks, so skipped calls never prompt and denied calls never appear as executed rows; only work that truly ran lands in Exec Report.
- The round-trip is fail-safe: browser disconnect, emit failure, cancel, or broker shutdown all resolve to Deny, so an unconfirmed state-changing action never slips through just because the UI vanished at the wrong time.

Windows attention issuing

- The current Windows attention path is explicit and local: the browser calls `POST /agent/flash_window/`, and the backend routes that request through `agent/window_flash.py`.
- That helper can flash the `Tlamatini.exe` console/taskbar window with `FlashWindowEx` and always prints an uppercase attention banner, so the signal survives in `tlamatini.log` even when the browser is minimized.
- Two concrete reasons are wired today: Ask Execs execution approval prompts and Notifier notifications. The path is best-effort and fail-safe, degrading cleanly on non-Windows or windowless launches.

Windows Installed-apps registration

- Introduced in `v1.11.0` and still carried by the current `v1.26.0` release, the frozen install now behaves like a real Windows application: `install.py` writes a per-user HKCU Add/Remove Programs entry so Tlamatini appears in Settings -> Apps -> Installed apps and in the legacy Programs and Features list.
- The entry carries `DisplayName`, `DisplayVersion`, `InstallLocation`, `DisplayIcon`, `UninstallString`, `QuietUninstallString`, `NoModify`, `NoRepair`, and best-effort `EstimatedSize`, all pointing at the bundled `Uninstaller.exe` without requiring administrator rights.
- The matching runtime self-heal in `agent/apps.py` calls `windows_app_registration.self_heal_for_frozen()` on every frozen launch, so installs created before this feature existed can appear in Windows' uninstall UI after the next normal app start.

De-Compresser agent

- De-Compresser is the deterministic archive worker for compression and decompression tasks, deciding direction from whichever side exposes a recognized archive extension.
- Supported archive families are `.gz`, `.zip`, `.7z`, `.tar.gz`, and `.gz.tar`; file-to-folder extraction and file-or-directory packing are both documented in the README and Book.
- Password handling is explicit: `passwordless=true` skips it, while `passwordless=false` requires the `DE_COMPRESSER_PWD` environment variable and fails fast if the secret is missing.

De-Compresser integration and fallback behavior

- The agent is reachable from both operator surfaces: ACP canvas nodes wire through dedicated connection-update views, and checked Multi-Turn can invoke it through `chat_agent_de_compresser`.
- Format engines are practical rather than magical: stdlib `gzip` and `zipfile` cover core cases, `7z` is preferred for encrypted `7z` or `zip`, and `py7zr` was added to `requirements.txt` as the Python fallback.
- Every run emits an `INI_SECTION_DE_COMPRESSER<<< ... >>>END_SECTION_DE_COMPRESSER` block and still triggers `target_agents`, so downstream Parametrizer or Raiser logic can branch on `success=true|false` instead of guessing from prose.

Unreal MCP and the Unrealer agent

- Unreal MCP is a UE5 plugin that runs inside the editor and listens on `127.0.0.1:55557` for one JSON command per TCP connection; Tlamatini is the client side of that link, not the plugin host.
- The Unrealer workflow agent forwards any command the connected plugin build exposes, up to a 53-command surface across nine categories: actor manipulation (incl. viewport screenshots), Blueprint authoring and node wiring, input mappings, UMG widget building, in-editor Python/console execution, level I/O, asset import, and material authoring.

Headless build/cook/test is out of scope (it needs UnrealEditor-Cmd, not the editor socket).

- The same integration is available in both operator surfaces: checked Multi-Turn via `chat_agent_unrealer`, and ACP canvas flows via the visual Unrealer node plus Parametrizer chaining.

Extended Unreal MCP surface

- Unrealer's documentation now points at the public `XAIHT/XaihtUnrealEngineMCP` fork, the Unreal Engine MCP variant developed specifically for Tlamatini.
- That fork exposes the extended 53-command, nine-category surface documented in README and Book: not only the base editor/Blueprint/UMG verbs, but also system, level, asset, material, screenshot, viewport, and in-editor Python paths.
- The practical message for operators is simple: Unrealer forwards the connected plugin's verb surface directly, so Tlamatini's client does not need a new release every time the plugin adds another supported command.

Installing the UE5 plugin and smoke-testing it

- Install the upstream plugin into `<YourProject>/Plugins/UnrealMCP/`, enable it from `Edit -> Plugins`, restart UE5, and confirm the Output Log says `UnrealMCP listening on 127.0.0.1:55557`.
- Tlamatini does not compile or embed the plugin; the Unreal editor must already be running with the listener bound before any Unrealer call can succeed.
- A seeded smoke test ships in the Prompts table: `Unreal MCP End-to-End Editor Drive` walks through sanity-check, actor spawn, Blueprint creation/compile, and UMG widget assembly.

Orphan-process cleanup

- Tlamatini now ships a three-tier orphan reaper in `Tlamatini/agent/orphan_reaper.py` focused on Windows console-host leftovers such as `conhost.exe` and `openconsole.exe`.
- Tier 1 runs after spawn-capable Multi-Turn tool calls, Tier 2 runs once after the final answer in a background thread, and Tier 3 runs again during application shutdown through the same cleanup path that already tears down pools.
- The candidate set stays narrow on purpose: dead descendants of the current process tree, orphaned console hosts tied to that tree, and pool-linked processes whose `cmdline` still points into `agents/pools/...` even though tracking records are gone.

Orphan-process prevention and survivor reporting

- Prevention landed alongside cleanup: Windows spawn sites now use `CREATE_NO_WINDOW | DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP` with stdio piped to `DEVNULL`, so the console host is usually never allocated in the first place.
- The ACPX runtime now kills full process trees instead of only top-level wrappers, and pool-agent scripts gained a conservative `subprocess.Popen` guard so forgotten descendants still inherit `CREATE_NO_WINDOW`.
- If something survives both cleanup tiers, Tlamatini sends a second chat bubble listing each surviving `name + PID`; the reaper never raises into the user path because a cleanup crash would be worse than the leftovers it tried to remove.

How to use it

- Run from source: create a virtual environment, install requirements, migrate, create a superuser, collect static files, and start Django.
- Open `/agent/` for chat. Load a file or directory context before asking codebase-specific questions.
- Keep Multi-Turn unchecked for direct Q&A; enable Multi-Turn for tasks that need tools, wrapped agents, monitoring, or workflow seeding.
- Use `chat_agent_globber` to find files by pattern, `chat_agent_grepper` to locate matching content, and `chat_agent_editor` when you need an exact in-place change instead of rewriting a whole file or shelling out to grep/findstr/sed.
- To use the External MCP capability, open `External -> MCPs`, register or import a server into `external_mcps.json`, choose the transport/runtime fields, and let the dialog connect it before expecting its `ext__<server>__<tool>` tools to appear in Multi-Turn.

- When you are onboarding or debugging an external MCP, call `chat_agent_mcp_doctor` first or use the MCP Doctor workflow node; it can tell you whether the issue is transport selection, a missing runtime on PATH, placeholder secrets, or a bad endpoint before you spend time on a live connect attempt.
- If you want a guided external-MCP onboarding flow, use the Step-by-Step mode in the External MCP dialog so each required field is introduced progressively instead of dumping the whole connection contract at once.
- Use Config -> Access Keys Wizard when you need to wire or update provider credentials without editing `config.json` manually.
- Use About -> Check for updates on packaged installs when you want Tlamatini to fetch and stage the latest release without manually replacing the install folder.
- Tick `Ask Execs` when you want human approval before each state-changing Multi-Turn step; it is disabled until Multi-Turn is on, and a single Deny stops the whole chain with an explicit red interruption banner.
- When you are using a packaged install on Windows 10 or Windows 11, uninstall it through Settings -> Apps -> Installed apps or the legacy Programs and Features entry, not by manually deleting the folder.
- If an Ask Execs approval dialog or a Notifier event needs you while the browser is buried, watch for Tlamatini's taskbar-attention flash and the matching uppercase banner in `tlamatini.log`.
- If you want her to inspect or modify herself, verify that `TlamatiniSourceCode/` exists in the current build first; self-modify is optional and absent builds must be treated honestly as read-only about their own code tree.
- For authorized Kali Linux assessments, run MCP-Kali-Server on the Kali box, set `Config -> URLs -> Kali server (Kalier)` once, and then call `chat_agent_kalier` from Multi-Turn with the desired `action` and `target` without repeating the box URL each turn.
- For STM32 firmware work, install STM32CubeIDE, leave `Config -> URLs -> STM32 MCP server script` blank for zero-config bootstrap, and then call `chat_agent_stm32er` from Multi-Turn with one `action` at a time such as `validate`, `create_project`, `write_source`, `build`, `build_and_flash`, `serial_session`, or `live_monitor`.
- For ESP32 firmware work, leave `Config -> URLs -> pio_executable` blank for zero-config PlatformIO bootstrap, then call `chat_agent_esp32er` from Multi-Turn with actions like `bootstrap`, `validate`, `create_project`, `write_source`, `build`, `upload`, `build_and_upload`, `monitor`, or `monitor_session`.
- For ESPHome smart-home firmware work, leave `Config -> URLs -> esphome_executable` blank for zero-config bootstrap, then call `chat_agent_esphomer` from Multi-Turn with actions like `bootstrap`, `validate`, `new_config`, `config`, `compile`, `upload`, `logs`, or the one-shot `scaffold_compile_upload` flow.
- For desktop-window control, call `chat_agent_windower` from Multi-Turn to focus, tile, resize, list, or close a window by title, or model the same action in ACP with the Windower node.
- For interactive web automation, call `chat_agent_playwrighter` from Multi-Turn with a `steps_json` script, or author the same step list visually with the Playwrighter node on the canvas.
- For Blender work, enable the official Blender MCP add-on in Blender, make sure its TCP listener is reachable (default `localhost:9876`), then call `chat_agent_blenderer` from Multi-Turn or use the Blenderer node on the canvas.
- For Unreal Engine work, enable the Unreal MCP plugin inside a live UE5 project first, then call `chat_agent_unrealer` from Multi-Turn or use the visual Unrealer node on the canvas.
- Archive jobs can now be described directly in Multi-Turn or modeled visually in ACP: De-Compresser infers compress vs decompress from the `input` or `output` extension.
- If a second post-answer warning bubble ever lists surviving `name + PID` entries, treat it as an honest cleanup report and end the listed processes manually from Task Manager if needed.
- Use the `ACPX-Skills` navbar menu when you need to browse, enable/disable, diagnose, or reload the shipped SKILL.md catalog without asking the LLM to be your admin surface.
- Use the DB dropdown when you need a safe database snapshot or want to stage a different `db.sqlite3` for the next start-up without hot-swapping the live SQLite file.
- Open `/agentic_control_panel/` to drag agents, connect them, configure each node, validate, start, pause/resume, stop, and save `.flw` workflows.
- Use `python build.py`, `python build_uninstaller.py`, and `python build_installer.py` only when producing a packaged Windows release.

Agent descriptions and catalog source of truth

- The authoritative human-readable source for workflow-agent descriptions is `agents_descriptions.md` at the repo root, not an embedded JavaScript map or a hard-coded Django list.
- Current validation compares live template directories, `agents_descriptions.md` rows, and the README / Book bestiary counts so the generated dossier stays aligned with the running product.
- The ACP sidebar hover tooltip and the right-click Description dialog both resolve through that markdown file first; `README.md` is only a legacy fallback when the dedicated description file is absent.

Reviewer and Analyzer

- The workflow catalog now includes two new high-value specialists: Reviewer and Analyzer, lifting the canvas inventory to 64 agents and the seed-skill catalog to 23 SKILL.md packages.
- Reviewer is LLM-powered: it resolves a git diff for `repo_path`, reviews it with a senior-engineer prompt, and emits an `INI_SECTION_REVIEWER` block whose first routable field is `verdict = APPROVE | REQUEST_CHANGES | COMMENT`.
- Analyzer is deterministic and scanner-driven: it runs whichever of `bandit`, `semgrep`, `ruff`, `eslint`, `gitleaks`, and `pip-audit` are installed on PATH over `target_path`, then emits an `INI_SECTION_ANALYZER` block with `status` and `total_findings` for downstream routing.
- Both agents always trigger `target_agents`, so a downstream Forker can branch on `{verdict}`, `{status}`, or `{total_findings}` instead of scraping prose.
- They are intentionally canvas-only workflow agents: there is no wrapped `chat_agent_reviewer` or `chat_agent_analyzer`, and therefore no duplicate Exec Report row family for those names.
- The chat-side counterparts live in the ACPX skill catalog instead: `code-review` exposes senior-engineer git-diff review, while `security-audit` exposes the deterministic multi-scanner sweep.

Operator surface counts

- The live operator surface now stands at 82 workflow agents, 89 Multi-Turn tools, 12 ACPX tools, and 27 skills.
- Source inspection confirms the total: 57 distinct wrapped chat-agent tools bound from `chat_agent_registry.py`, which combines with 20 core Python tools and 12 ACPX/Skill tools for 89 Multi-Turn tools overall.
- The count growth over older public badges now comes from several stacked waves together: the deterministic file-navigation/file-edit trio (Globber, Grepper, Editor), the ESPHomer smart-home firmware lane, and the newer MCP Doctor onboarding specialist that accompanies the External MCP universal client release.
- The workflow-agent and wrapped-tool totals are validated from the live tree even when some handbook badges or older prose lines lag behind the newest release wave, so the dossier stays tied to source truth instead of stale summaries.
- This matters operationally because the planner never binds everything at once: the documented default `max_selected_tools` cap stays at 20, so breadth of capability does not mean uncontrolled tool sprawl per turn.

Kalier current role

- Kalier is Tlamatini's current Kali Linux bridge: a direct client for MCP-Kali-Server that lets her drive authorized recon, enumeration, web scanning, and offensive-security workflows from chat or canvas.
- It can issue one capability per run, including `nmap`, `gobuster`, `dirb`, `nikto`, `sqlmap`, `metasploit`, `hydra`, `john`, `wpscan`, `enum4linux`, arbitrary `command`, or a safe `health` probe of the remote server.
- The agent emits `INI_SECTION_KALIER` with `action`, `endpoint`, `subject`, `return_code`, `success`, `timed_out`, and `server_url`, so downstream Forker or Parametrizer logic can branch on results without scraping prose.
- The practical operator result is simpler prompts: after the Kali box URL is configured once, normal Multi-Turn requests no longer repeat `server_url` unless you intentionally override it for a one-off target.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_kalier` now auto-injects the configured `kali_server_url`, while the visual Kalier canvas node still stores `server_url` explicitly in YAML per node.
- Kalier talks straight to the Kali-side Flask API over HTTP using Python-stdlib `urllib`, so it stays self-contained in the pool subprocess and works the same in source or frozen builds; the embedded-client injection fails open if the config value is blank or unreadable.

- Authorized use only: the intended operator flow is an in-scope lab, CTF, or permitted engagement, often with `ssh -L 5000:localhost:5000 user@KALI_IP` tunneling a remote Kali box back to `http://127.0.0.1:5000`, and the repo now includes `Tlamatini-Kali-Setup.md` for the zero-client walkthrough.

STM32er current role

- STM32er is Tlamatini's current STM32 bridge: she talks to the `STM32 Template Project MCP` so she can scaffold, author, build, flash, reset, and observe STM32F4 firmware without driving STM32CubeIDE manually.
- The operator promise is zero-config bootstrap: leave `server_script` blank and STM32er downloads or zip-falls-back to the MCP server on first use, installs `mcp` and `pyserial` if needed, validates, and caches the result so the user only installs STM32CubeIDE plus Tlamatini.
- Before any compile-or-hardware action, STM32er runs a critical-mission fail-safe preflight over the arm-none-eabi toolchain, STM32CubeIDE, programmer path, ST-LINK probe, and target family; compile-only steps can run boardless, but flash, erase, reset, serial, and SWD/live-memory operations are refused when the environment or device is wrong.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_stm32er` takes one `action` per call, while the visual STM32er canvas node stores the same fields in YAML and triggers downstream agents on both success and failure.
- The tool surface includes the full project lifecycle plus hardware-in-the-loop composites: `validate`, `bootstrap`, `create_project`, `write_source`, `build`, `build_and_flash`, `serial_session`, `live_monitor`, and the rest of the 23 MCP verbs, with every run emitting an `INI_SECTION_STM32ER` block for Forker or Parametrizer routing.
- Config -> URLs now seeds the chat path with `stm32_mcp_server_script`, `stm32_mcp_python`, `stm32_template_dir`, `stm32_ide_root`, `stm32_mcp_repo_url`, and `stm32_mcp_install_dir`, so firmware prompts normally describe only the task and target board instead of the plumbing.

ESP32er current role

- ESP32er is Tlamatini's current PlatformIO Core bridge: she can scaffold, author, build, upload, and monitor ESP32-class firmware without relying on an external MCP server or IDE.
- The operator promise is zero-config bootstrap: leave `pio_executable` blank and ESP32er downloads or pip-falls-back to PlatformIO Core on first use, validates it, and caches it under a per-user directory so the user installs only the board USB driver plus Tlamatini.
- Before any build-or-upload action, ESP32er runs a serial-aware preflight over `pio` resolution, project shape, and connected ports; upload and monitor require a real serial device, while non-espressif32 targets are warned about rather than hard-refused because PlatformIO is intentionally multi-target.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_esp32er` takes one `action` per call, while the visual ESP32er canvas node stores the same fields in YAML and triggers downstream agents on both success and failure.
- The tool surface covers environment/meta (`bootstrap`, `validate`, `system_info`, `boards`), project lifecycle (`create_project`, `write_source`, `read_source`, `list_sources`, `clean`), build and flash (`build`, `upload`, `build_and_upload`, `list_artifacts`), serial HIL (`device_list`, `monitor`, `monitor_session`), and package / QA paths (`pkg_install`, `pkg_list`, `pkg_update`, `check`, `test`), with every run emitting an `INI_SECTION_ESP32ER` block for Forker or Parametrizer routing.
- Config -> URLs now seeds the chat path with `pio_executable` and `pio_core_dir`, so firmware prompts usually describe only the board, project, and task while the wrapped tool auto-injects the PlatformIO runtime plumbing.

ESP32 Template Project reference baseline

- The new `ESP32TemplateProject` repository is the known-good baseline documented in BookOfTlamatini's bonus chapter: a plain PlatformIO project, not a server, meant to prove an ESP32 board and toolchain are healthy before larger firmware work.
- It mirrors ESP32er's grain: `platformio.ini`, `src/`, `include/`, `lib/`, `test/`, a blinking `main.cpp`, serial output at 115200, and GitHub-ready docs/CI so the reference project does not silently rot.
- Operationally, ESP32er can either point at a checkout of that template (`project_dir`) or scaffold an equivalent from scratch with `action='create_project'`, then carry the directory through build, upload, and monitor steps.

ESPHomer current role

- ESPHomer is Tlamatini's ESPHome bridge: she can author YAML-based smart-home device configs, validate them, compile firmware, upload over USB or OTA, and observe logs without introducing an extra MCP server or IDE.
- The operator promise is zero-config bootstrap: leave `esphome_executable` blank and ESPHomer installs or resolves ESPHome on first use, so the user installs only the board USB driver plus Tlamatini.
- Before any compile-or-upload action, ESPHomer runs a fail-safe preflight over `esphome` resolution, YAML existence, and serial-or-OTA readiness; upload, run, and logs require a real serial board or an OTA host because the first flash is always USB.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_esphomer` takes one `action` per call, while the visual ESPHomer canvas node stores the same fields in YAML and triggers downstream agents on both success and failure.
- The tool surface covers environment/meta (`bootstrap`, `validate`, `version`), device-YAML lifecycle (`new_config`, `write_config`, `read_config`, `config`, `clean`), build and flash (`compile`, `upload`, `run`, `list_artifacts`), bounded observation (`logs`), and the one-shot `scaffold_compile_upload` lifecycle.
- Config -> URLs now seeds the chat path with `esphome_executable`, so smart-home firmware prompts usually describe only the device, board, and task while the wrapped tool auto-injects the ESPHome runtime plumbing.

ESPHome template baseline

- The bundled `ESPHomeTemplateProject` is the known-good ESPHome baseline documented in README and Book: a phone-controllable light with native `api`, `ota`, `wifi`, and a board LED output, shipped as `agent/agents/esphomer/ESPHomeTemplateProject/tlamatini-light.yaml`.
- It matters because ESPHomer's source-of-truth is a YAML device file, not a C++ project tree: that sample proves the first real workflow of `new_config` -> `config` -> `compile` -> `upload` without forcing users to invent a valid starter config from memory.
- Operationally, the sample closes the gap between the new agent and a practical first build a user can validate, compile, flash, and then control from a smart-home hub such as Home Assistant.

Windower on Multi-Turn and canvas

- Windower is the new 66th workflow agent: a deterministic Win32 window manager that finds an application window by title and performs one lifecycle operation on the window itself instead of clicking inside it.
- It supports `focus`, `minimize`, `maximize`, `restore`, `move`, `resize`, `move_resize`, `close`, `topmost`, `untopmost`, `arrange`, and `list`, with `substring`, `exact`, or `regex` title matching plus `match_index` for duplicate titles.
- The agent emits `INI_SECTION_WINDOWER` with `action`, `window_title`, `matched`, `match_count`, `state`, `left`, `top`, `width`, and `height`, so downstream Forker or Parametrizer logic can branch on presence, state, or geometry.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_windower` accepts free-form key=value requests, while the visual Windower canvas node stores the same operation fields in YAML.
- Windower is the desktop-window sibling of Mouser and Keyboarder: use Windower when the goal is the window as a whole, Mouser for controls inside it, and Keyboarder for text entry into it.
- Because Windower changes real window state, its executions appear in Exec Report and it always triggers `target_agents` whether the action succeeds or fails.

Playwrighter current role

- Playwrighter is the new 65th workflow agent in `v1.5.0`: a deterministic Playwright-powered browser automator for Chromium, Firefox, or WebKit that executes ordered interactive steps instead of static fetches.
- It covers the gap between Crawler and Googler: logins, multi-step forms, wizard clicks, JS-rendered SPA scraping, screenshots after interaction, assertions, downloads, and authenticated end-to-end UI checks.
- The agent emits `INI_SECTION_PLAYWRIGHTER` with `start_url`, `final_url`, `status`, `steps_run`, `assert_result`, and extracted values so downstream Forker or Parametrizer logic can branch on pass/fail or reuse scraped data.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_playwrighter` takes the whole script as `steps_json`, while the visual Playwrighter canvas node stores the same declarative step list in YAML.

- Session continuity is built in: `headless: false` lets operators watch it drive, and `storage_state_in` / `storage_state_out` carry login state across runs without forcing a manual browser setup each time.
- Because Playwrighter is state-changing, its executions appear in Exec Report and it always triggers `target_agents` whether the run succeeds or fails.

Reviewer precision patch in v1.4.1

- The `v1.4.1` Reviewer refinement tightened accuracy rather than adding a new surface: when `diff_ref` is empty, the review prompt labels the diff as the uncommitted working tree plus staged area, so the model must not describe those findings as already committed or pushed.
- The same patch teaches the model Tlamatini's managed-secret convention: `agent/config.json` and selected `agent/agents/*/config.yaml` files can hold live local credentials in a keyed working copy, while `regen_secrets.py --mode push-able` scrubs them back to placeholders before commit.
- That guidance is mirrored into the `code-review` SKILL.md package too, keeping the chat-surface review behavior and the canvas Reviewer agent aligned on commit-state wording and secret-severity expectations.

Native dialogs and Tkinter removal in v1.4.2

- The current tagged release `v1.4.2` removes Tkinter from the unstable runtime-facing dialog path and replaces it with `Tlamatini/agent/native_dialogs.py`, a native Windows dialog bridge used by browser-triggered pickers.
- This change pairs with the existing DB and operator dialogs: file and folder selection still feels local and GUI-first, but the fragile Tkinter dependency is no longer part of the interactive runtime path that users trigger from chat or ACP surfaces.
- The patch arrived with dedicated tests (`test_native_dialogs.py`) and with follow-up orphan-reaper/runtime adjustments, so the release reads as a stability pass rather than a cosmetic refactor.

ACPX-Skills menu

- The new `ACPX-Skills` navbar menu gives operators four direct actions over the skill catalog: Browse Skills, Configure Skills, Diagnostics, and Reload Registry.
- `Configure Skills` flips `Skill.enabled` exactly the way MCPs and Tools are toggled, so disabled skills disappear from `list_skills` and reject `invoke_skill` with `SKILL_DISABLED` instead of silently half-working.
- The diagnostics view cross-checks skill dependencies against disabled tools, disabled MCPs, missing ACPX agents, and orphan database rows whose SKILL.md disappeared from disk.

Source-mode bootstrap commands

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python Tlamatini/manage.py migrate
python Tlamatini/manage.py createsuperuser
python Tlamatini/manage.py collectstatic --noinput
python Tlamatini/manage.py runserver --noreload
```

4. Ollama Setup Without Administrative Rights

- Open a normal PowerShell window, not an elevated one, for the safest no-admin Windows installation path.
- Install into `%LOCALAPPDATA%\Programs\Ollama` with the official PowerShell installer script and then reopen PowerShell so PATH updates are visible.
- If you plan to use the default `:cloud` models from the shipped config, sign in with `ollama signin` so the host is linked to your Ollama account before you test the app.
- Verify the CLI with `ollama --version`, start `ollama serve` if the background service is not already active, and confirm `http://127.0.0.1:11434/api/tags` responds.
- Pull the default repository model tags exactly as written if you want the shipped config and agent templates to work unchanged.
- The Book now clarifies the token rule: a localhost Ollama usually needs no Ollama bearer token in Tlamatini, while a remote Ollama endpoint may require one in `Config -> Access Keys Wizard` or the matching config key.

No-admin Ollama commands and default model pulls

```
$env:OLLAMA_INSTALL_DIR = "$env:LOCALAPPDATA\Programs\Ollama"
irm https://ollama.com/install.ps1 | iex
ollama --version
ollama signin
ollama serve
Invoke-WebRequest http://127.0.0.1:11434/api/tags -UseBasicParsing
ollama pull Nomic-Embed-Text:latest
ollama pull kimi-k2.6:cloud
ollama pull qwen3.5:cloud
ollama pull gpt-oss:120b-cloud
ollama pull qwen3.5:397b-cloud
ollama pull glm-5.1:cloud
```

5. Runtime, Release, and Operator Diagnostics

Running the application

- Development server: `python Tlamatini/manage.py runserver --noreload`.
- Preferred async/dev bootstrap: `python Tlamatini/manage.py startserver`, which starts MCP services before the Django server.
- Production-style ASGI entrypoint: `daphne -b 127.0.0.1 -p 8000 tlamatini.asgi:application`.
- Current startup also re-applies GPU-performance / Ollama-pinning hooks in the background on supported NVIDIA Windows hosts, so restart-time behavior stays closer to the tuned development baseline.
- That same early startup window is also where a staged `DB/ToLoad/db.sqlite3` file is promoted into the live database path, before Django opens SQLite.
- In frozen mode, startup now also self-heals the per-user Windows Installed-apps entry when `Uninstaller.exe` is present beside `Tlamatini.exe`, so old installs can gain a standard uninstall surface without a reinstall.
- Startup now also prints a `--- [VERSION] Tlamatini ...` banner, making the running build visible in both the console and `tlamatini.log` without an HTTP call.
- Startup cleans pool state, repopulates the agent registry, launches MCP metrics/file-search servers, and then serves HTTP plus WebSocket traffic.

Embedding-memory pre-flight guard

- README and BookOfTlamatini now document an embedding-memory pre-flight guard for GPU hosts before a directory-context load starts its FAISS embedding burst.
- On supported NVIDIA hosts it estimates embedding-model VRAM pressure, and when the projected load is too high it emits a non-blocking warning chat bubble instead of silently letting RAM<->VRAM thrash surprise the operator.
- CPU-only, AMD, and Apple-Silicon hosts stay fail-open: the guard becomes a no-op when the NVIDIA probe does not apply.
- The practical operator response is straightforward: switch to a smaller embedding model, reconnect if needed, or proceed knowingly with the heavier model.

Reconnect and restart safeguards

- Config -> Models and Config -> URLs dialogs now track their pre-edit baseline and can show a reconnect-required dialog when the saved values change what the live chat session should trust.
- The restored-session autoload path now buffers early WebSocket frames so context-loading spinners and disabled-input state are not lost during automatic reconnect/restore flows.
- Startup and restart behavior now also re-apply GPU performance and Ollama keep-alive hooks in the background on supported NVIDIA Windows hosts, improving warm-model readiness without blocking Django boot.
- Browser-driven attention routing is now part of that operator-safety layer too: Ask Execs prompts and Notifier events can raise a taskbar flash plus a log banner without relying on the browser window already being visible.
- Windows process hygiene is now part of that safety story too: detached no-window spawns and the three-tier orphan reaper reduce the chance that Task Manager shows stale Tlamatini-icon console helpers after long runs.
- Ask Execs extends that safety story into execution approval itself: the operator can now stop a destructive chain before the next mutation instead of only auditing it after the fact in Exec Report.

Media and voice family

- The current media family spans capture, playback, and speech: Shoter (screen), Camcorder (camera-in), Recorder (mic-in), AudioPlayer (speakers-out), VideoPlayer (screen-out with audio), Talker (text-to-speech), and Whisperer (speech-to-text).
- Talker remains female-only by design and uses an Ollama neural TTS model with SNAC decoding, while Whisperer records the microphone itself or transcribes a file through faster-whisper locally or cloud Whisper APIs.

- Recent stability work matters here too: Talker now chunks long input by sentence for long-form speech, and media-output defaults were moved into the application `Temp` directory rather than user content folders.

Autonomous command watchdog

- A boot-time autonomous command watchdog now protects the chat against shell wrappers that stay alive while making zero CPU or I/O progress, the classic signature of a mangled prompt waiting forever on stdin.
- It never kills on elapsed time alone: the subtree must outlive the grace window and remain idle for a configured streak, so long builds, downloads, and compiles are preserved while only genuinely wedged interpreters are reaped.
- This watchdog complements rather than replaces the orphan reaper: the watchdog rescues blocked synchronous tool calls before they return, while the orphan reaper still handles post-return survivors and stale console companions.

Prompt catalog and answer readability discipline

- Version `1.3.2` tightened the HTML answer contract with a Prime Directive on visual readability: explicit background and text color, no grey-on-dark body text, and safer table-body defaults.
- The seeded `Prompts` dropdown was also re-sorted into a learner path: context-only Q&A first, then metrics, files search, shell, code generation, vision, specialized single-tool actions, agent control, Unrealer, and heavier Multi-Turn/ACPX demos last.
- Those readability rules remain in force in the current documentation set, and the current `v1.26.0` release state keeps the version badge, runtime surfaces, self-knowledge wording, STM32er/ESP32er demo prompts, and operator handbook aligned.

Ask Execs and execution approval

- Introduced in `v1.10.0` and still part of the current `v1.26.0` surface, `Ask Execs` is the Multi-Turn-only safety modifier that makes Tlamatini ask before each state-changing Tool, MCP, wrapped agent, or skill-backed execution instead of running it immediately.
- The permission dialog is explicit and auditable: it names the Tool or Agent family, the underlying raw tool name, the full parameters, the program or command to be executed, and the shell or execution surface involved.
- Proceed runs that one step and then prompts again at the next state-changing step; Deny halts the entire chain immediately and appends a red `Execution interrupted` banner even when Exec Report itself is off.
- Under the hood, `agent/exec_permission.py` provides `ExecPermissionBroker`, which lets the synchronous worker-thread executor emit a permission request onto the WebSocket event loop and then block on a `threading.Event` until the browser replies.
- The gate sits after deduplication and quota checks, so skipped calls never prompt and denied calls never appear as executed rows; only work that truly ran lands in Exec Report.
- The round-trip is fail-safe: browser disconnect, emit failure, cancel, or broker shutdown all resolve to Deny, so an unconfirmed state-changing action never slips through just because the UI vanished at the wrong time.

Windows attention issuing

- The current Windows attention path is explicit and local: the browser calls `POST /agent/flash_window/`, and the backend routes that request through `agent/window_flash.py`.
- That helper can flash the `Tlamatini.exe` console/taskbar window with `FlashWindowEx` and always prints an uppercase attention banner, so the signal survives in `tlamatini.log` even when the browser is minimized.
- Two concrete reasons are wired today: Ask Execs execution approval prompts and Notifier notifications. The path is best-effort and fail-safe, degrading cleanly on non-Windows or windowless launches.

Unreal MCP runtime behavior

- Each Unrealer run is one command: load `config.yaml`, open a fresh TCP socket, send `{"type": command, "params": params}`, read until valid JSON arrives, and log one atomic `INI_SECTION_UNREALER<<< ... >>>END_SECTION_UNREALER` block.
- The wrapped-tool path stores chat runs under `agent/agents/pools/_chat_runs_/unrealer_<seq>_<id>/`, while visual flows use normal `unrealer_<n>` pool folders and can chain response fields through Parametrizer mappings.

- Exec Report treats Unrealer as its own row family, and troubleshooting stays concrete: connection-refused means the plugin is not listening, while read-timeout usually means UE5's game thread is busy.

Orphan reaper runtime behavior

- Tlamatini now ships a three-tier orphan reaper in `Tlamatini/agent/orphan_reaper.py` focused on Windows console-host leftovers such as `conhost.exe` and `openconsole.exe`.
- Tier 1 runs after spawn-capable Multi-Turn tool calls, Tier 2 runs once after the final answer in a background thread, and Tier 3 runs again during application shutdown through the same cleanup path that already tears down pools.
- The candidate set stays narrow on purpose: dead descendants of the current process tree, orphaned console hosts tied to that tree, and pool-linked processes whose `cmdline` still points into `agents/pools/...` even though tracking records are gone.
- Prevention landed alongside cleanup: Windows spawn sites now use `CREATE_NO_WINDOW | DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP` with stdio piped to `DEVNULL`, so the console host is usually never allocated in the first place.
- The ACPX runtime now kills full process trees instead of only top-level wrappers, and pool-agent scripts gained a conservative `subprocess.Popen` guard so forgotten descendants still inherit `CREATE_NO_WINDOW`.
- If something survives both cleanup tiers, Tlamatini sends a second chat bubble listing each surviving `name + PID`; the reaper never raises into the user path because a cleanup crash would be worse than the leftovers it tried to remove.

Release pipeline

- Release production is a three-step pipeline: `build.py` -> `build_uninstaller.py` -> `build_installer.py`.
- The final distributable is the full versioned release folder `dist/Tlamatini_Release_v<version>/`, not a stray executable copied outside its payload.
- Use `build.py --self-modify` when you intentionally want a release that ships `TlamatiniSourceCode/` so she can inspect or modify herself at runtime.
- Current `build.py` treats `README.md` and `jd-cli/` as required post-build assets and fails hard if those payloads are missing.
- Bundled support scripts cover shortcut creation/removal, `.flw` association, the PowerShell launcher, Windows-specific installer ergonomics, and the per-user Installed-apps registration path.

Windows uninstall registration

- Introduced in `v1.11.0` and still carried by the current `v1.26.0` release, the frozen install now behaves like a real Windows application: `install.py` writes a per-user HKCU Add/Remove Programs entry so Tlamatini appears in Settings -> Apps -> Installed apps and in the legacy Programs and Features list.
- The entry carries `DisplayName`, `DisplayVersion`, `InstallLocation`, `DisplayIcon`, `UninstallString`, `QuietUninstallString`, `NoModify`, `NoRepair`, and best-effort `EstimatedSize`, all pointing at the bundled `Uninstaller.exe` without requiring administrator rights.
- The matching runtime self-heal in `agent/apps.py` calls `windows_app_registration.self_heal_for_frozen()` on every frozen launch, so installs created before this feature existed can appear in Windows' uninstall UI after the next normal app start.

Exec Report

- Exec Report is a Multi-Turn-only transparency layer that appends one operation table per state-changing agent family to the final answer.
- Rows are recorded from the live tool-call stream rather than guessed from the LLM prose, so the report is the operational ground truth.
- Each row receives a SUCCESS/FAILURE verdict from raw tool returns, making long installs, deployments, and remediations inspectable after the fact.
- When Ask Execs is enabled, Exec Report and the red denial banner complement each other: already-executed steps still render as tables, while the denied step stays out of the tables and is surfaced only through the interruption banner.

ACPX and skills

- ACPX lets Tlamatini spawn external coding-agent CLIs such as Codex, Claude Code, Cursor, Gemini, Qwen, and others as managed child processes.
- It pairs those agents with markdown-driven `SKILL.md` packages, validated I/O contracts, permission gating, and append-only audit logs.
- Transport-aware drain rules and bounded event bodies reduce latency while protecting the LLM context budget during external-agent relays.
- The operator-facing references are `README.md` and `ACPX.md`, while the implementation lives under `agent/acpx/`, `agent/skills/`, and `agent/skills_pkg/`.

Application log

- The built-in `tlamatini.log` file captures both stdout and stderr through a tee stream initialized in `manage.py` before Django starts.
- In source mode the log sits next to `manage.py`; in frozen mode it lives next to the executable.
- Immediate flush behavior makes the log the primary forensic artifact for startup problems, warnings, tracebacks, and runtime diagnostics.

6. Agent Catalog and Runtime Model

Tlamatini currently exposes 82 workflow-agent templates.

Category	Representative agents
Control	starter, ender, stopper, cleaner, barrier, flowbacker
Execution and files	executer, pythonxer, pser, file_creator, file_extractor, file_interpreter, de_compresser, playwrighter, windower, unrealer, kalier, stm32er, esp32er, esphomer, arduiner, mover, deleter
DevOps and infra	gitter, dockerer, kuberneter, jenkinser, ssher, scper
Data and APIs	sqler, mongoxer, apirer, crawler, googler
Monitoring and routing	monitor_log, monitor_netstat, flowhypervisor, forker, asker, counter, and, or
Communication	notifier, emailer, recmailer, telegramer, telegramrx, teletlamatini, whatsapper, whatstlamatini
Security and media	kyber_keygen, kyber_cipher, kyber_decipher, image_interpreter, shoter, camcorder, recorder, audioplayer, videoplayer, j_decompiler
Workflow intelligence	flowcreator, gatewayer, gateway_relayer, node_manager, parametrizer, prompter, summarizer, acpxer

All workflow agents follow a common deployment pattern: template directory, YAML configuration, session-scoped pool copy, PID/status/log files, target/source wiring, and optional reanimation state.

Agent catalog validation

- The authoritative human-readable source for workflow-agent descriptions is `agents_descriptions.md` at the repo root, not an embedded JavaScript map or a hard-coded Django list.
- Current validation compares live template directories, `agents_descriptions.md` rows, and the README / Book bestiary counts so the generated dossier stays aligned with the running product.
- The ACP sidebar hover tooltip and the right-click Description dialog both resolve through that markdown file first; `README.md` is only a legacy fallback when the dedicated description file is absent.

How agent runtimes are shaped

- Every workflow agent follows the same operational skeleton: template directory, `config.yaml`, a session-scoped pool copy, PID/status/log files, and explicit source/target wiring.
- Chat-wrapped tool calls launch isolated runtime copies under `agent/agents/pools/_chat_runs_/`, while ACP uses named pool folders such as `starter_1` or `unrealer_1`.
- Specialized agents now stretch the platform in different directions: Globber/Grepper/Editor cover deterministic file discovery, regex search, and surgical in-place edits; MCP Doctor performs safe external-MCP onboarding diagnosis; ACPXer drives external coding-agent CLIs; Kalier drives a remote or tunneled Kali Linux tool server; STM32er drives a zero-config STM32 firmware MCP bridge; ESP32er drives PlatformIO directly; ESPHomer drives ESPHome directly for YAML-authored smart-home devices; Blenderer drives a live Blender editor over the official MCP add-on socket; Unrealer drives a live UE5 editor; and TeleTlamatini / WhatsTlamatini bridge full Tlamatini conversations into messaging platforms.

MCP Doctor spotlight

- MCP Doctor is the new onboarding and triage specialist added with the External MCP release: she inspects a configured server entry and tells the operator what is wrong before a live connection attempt wastes time.
- The diagnostic checks cover transport shape, runtime command availability on PATH, placeholder or missing secrets, endpoint/source/docs links, and the concrete next step the operator should take to make the server connectable.
- Operators can reach the same behavior from both surfaces that matter: the MCP Doctor workflow agent on the canvas and the wrapped `chat_agent_mcp_doctor` tool inside Multi-Turn.

Blenderer spotlight

- Blenderer was introduced in `v1.20.0` as the 77th workflow agent and remains the direct bridge to the official Blender MCP add-on, letting Tlamatini operate a live Blender session from either Multi-Turn chat or the visual workflow canvas.

- The bridge talks over a TCP socket (default `localhost:9876`) and supports both raw `execute_code` requests and higher-level scene/object/material/render actions, so operators can mix deterministic verbs with precise Python-driven 3D automation.
- This extends Tlamatini beyond code and firmware orchestration into DCC / 3D-production work: asset inspection, scene mutation, material changes, camera setup, and render-triggering now live inside the same operator surface as the rest of the system.

Self-update spotlight

- The packaged application now includes an in-app self-update flow exposed from About -> Check for updates, so operators can refresh a frozen install without manually unpacking and replacing the entire release folder.
- The backend checks the latest GitHub release, downloads and stages the package, then hands off the locked-file swap to `apply_update.ps1`, which performs the replacement after the running executable exits and relaunches the updated app.
- The update path is state-preserving by design: it keeps `config.json`, user content, and one `agents_backup` generation, and it stages the user's database through `DB/ToLoad/` plus `post_update_migrate.flag` so the next launch restores and migrates that DB instead of wiping it.

Reviewer and Analyzer spotlight

- The workflow catalog now includes two new high-value specialists: Reviewer and Analyzer, lifting the canvas inventory to 64 agents and the seed-skill catalog to 23 SKILL.md packages.
- Reviewer is LLM-powered: it resolves a git diff for `repo_path`, reviews it with a senior-engineer prompt, and emits an `INI_SECTION_REVIEWER` block whose first routable field is `verdict = APPROVE | REQUEST_CHANGES | COMMENT`.
- Analyzer is deterministic and scanner-driven: it runs whichever of `bandit`, `semgrep`, `ruff`, `eslint`, `gitleaks`, and `pip-audit` are installed on PATH over `target_path`, then emits an `INI_SECTION_ANALYZER` block with `status` and `total_findings` for downstream routing.
- Both agents always trigger `target_agents`, so a downstream Forker can branch on `{verdict}`, `{status}`, or `{total_findings}` instead of scraping prose.
- They are intentionally canvas-only workflow agents: there is no wrapped `chat_agent_reviewer` or `chat_agent_analyzer`, and therefore no duplicate Exec Report row family for those names.
- The chat-side counterparts live in the ACPX skill catalog instead: `code-review` exposes senior-engineer git-diff review, while `security-audit` exposes the deterministic multi-scanner sweep.

Operator surface counts

- The live operator surface now stands at 82 workflow agents, 89 Multi-Turn tools, 12 ACPX tools, and 27 skills.
- Source inspection confirms the total: 57 distinct wrapped chat-agent tools bound from `chat_agent_registry.py`, which combines with 20 core Python tools and 12 ACPX/Skill tools for 89 Multi-Turn tools overall.
- The count growth over older public badges now comes from several stacked waves together: the deterministic file-navigation/file-edit trio (Globber, Grepper, Editor), the ESPHomer smart-home firmware lane, and the newer MCP Doctor onboarding specialist that accompanies the External MCP universal client release.
- The workflow-agent and wrapped-tool totals are validated from the live tree even when some handbook badges or older prose lines lag behind the newest release wave, so the dossier stays tied to source truth instead of stale summaries.
- This matters operationally because the planner never binds everything at once: the documented default `max_selected_tools` cap stays at 20, so breadth of capability does not mean uncontrolled tool sprawl per turn.

Kalier spotlight

- Kalier is Tlamatini's current Kali Linux bridge: a direct client for MCP-Kali-Server that lets her drive authorized recon, enumeration, web scanning, and offensive-security workflows from chat or canvas.
- It can issue one capability per run, including `nmap`, `gobuster`, `dirb`, `nikto`, `sqlmap`, `metasploit`, `hydra`, `john`, `wpscan`, `enum4linux`, arbitrary `command`, or a safe `health` probe of the remote server.
- The agent emits `INI_SECTION_KALIER` with `action`, `endpoint`, `subject`, `return_code`, `success`, `timed_out`, and `server_url`, so downstream Forker or Parametrizer logic can branch on results without scraping prose.

- The practical operator result is simpler prompts: after the Kali box URL is configured once, normal Multi-Turn requests no longer repeat `server_url` unless you intentionally override it for a one-off target.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_kalier` now auto-injects the configured `kali_server_url`, while the visual Kalier canvas node still stores `server_url` explicitly in YAML per node.
- Kalier talks straight to the Kali-side Flask API over HTTP using Python-stdlib `urllib`, so it stays self-contained in the pool subprocess and works the same in source or frozen builds; the embedded-client injection fails open if the config value is blank or unreadable.
- Authorized use only: the intended operator flow is an in-scope lab, CTF, or permitted engagement, often with `ssh -L 5000:localhost:5000 user@KALI_IP` tunneling a remote Kali box back to `http://127.0.0.1:5000`, and the repo now includes `Tlamatini-Kali-Setup.md` for the zero-client walkthrough.

STM32er spotlight

- STM32er is Tlamatini's current STM32 bridge: she talks to the `STM32 Template Project MCP` so she can scaffold, author, build, flash, reset, and observe STM32F4 firmware without driving STM32CubeIDE manually.
- The operator promise is zero-config bootstrap: leave `server_script` blank and STM32er downloads or zip-falls-back to the MCP server on first use, installs `mcp` and `pyserial` if needed, validates, and caches the result so the user only installs STM32CubeIDE plus Tlamatini.
- Before any compile-or-hardware action, STM32er runs a critical-mission fail-safe preflight over the arm-none-eabi toolchain, STM32CubeIDE, programmer path, ST-LINK probe, and target family; compile-only steps can run boardless, but flash, erase, reset, serial, and SWD/live-memory operations are refused when the environment or device is wrong.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_stm32er` takes one `action` per call, while the visual STM32er canvas node stores the same fields in YAML and triggers downstream agents on both success and failure.
- The tool surface includes the full project lifecycle plus hardware-in-the-loop composites: `validate`, `bootstrap`, `create_project`, `write_source`, `build`, `build_and_flash`, `serial_session`, `live_monitor`, and the rest of the 23 MCP verbs, with every run emitting an `INI_SECTION_STM32ER` block for Forker or Parametrizer routing.
- Config -> URLs now seeds the chat path with `stm32_mcp_server_script`, `stm32_mcp_python`, `stm32_template_dir`, `stm32_ide_root`, `stm32_mcp_repo_url`, and `stm32_mcp_install_dir`, so firmware prompts normally describe only the task and target board instead of the plumbing.

Windower spotlight

- Windower is the new 66th workflow agent: a deterministic Win32 window manager that finds an application window by title and performs one lifecycle operation on the window itself instead of clicking inside it.
- It supports `focus`, `minimize`, `maximize`, `restore`, `move`, `resize`, `move_resize`, `close`, `topmost`, `untopmost`, `arrange`, and `list`, with `substring`, `exact`, or `regex` title matching plus `match_index` for duplicate titles.
- The agent emits `INI_SECTION_WINDOWER` with `action`, `window_title`, `matched`, `match_count`, `state`, `left`, `top`, `width`, and `height`, so downstream Forker or Parametrizer logic can branch on presence, state, or geometry.
- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_windower` accepts free-form key=value requests, while the visual Windower canvas node stores the same operation fields in YAML.
- Windower is the desktop-window sibling of Mouser and Keyboarder: use Windower when the goal is the window as a whole, Mouser for controls inside it, and Keyboarder for text entry into it.
- Because Windower changes real window state, its executions appear in Exec Report and it always triggers `target_agents` whether the action succeeds or fails.

Playwrighter spotlight

- Playwrighter is the new 65th workflow agent in `v1.5.0`: a deterministic Playwright-powered browser automator for Chromium, Firefox, or WebKit that executes ordered interactive steps instead of static fetches.
- It covers the gap between Crawler and Googler: logins, multi-step forms, wizard clicks, JS-rendered SPA scraping, screenshots after interaction, assertions, downloads, and authenticated end-to-end UI checks.
- The agent emits `INI_SECTION_PLAYWRIGHTER` with `start_url`, `final_url`, `status`, `steps_run`, `assert_result`, and extracted values so downstream Forker or Parametrizer logic can branch on pass/fail or reuse scraped data.

- Two operator surfaces ship in lock-step: the wrapped Multi-Turn tool `chat_agent_playwrighter` takes the whole script as `steps_json`, while the visual Playwrighter canvas node stores the same declarative step list in YAML.
- Session continuity is built in: `headless: false` lets operators watch it drive, and `storage_state_in` / `storage_state_out` carry login state across runs without forcing a manual browser setup each time.
- Because Playwrighter is state-changing, its executions appear in Exec Report and it always triggers `target_agents` whether the run succeeds or fails.

Reviewer precision spotlight

- The `v1.4.1` Reviewer refinement tightened accuracy rather than adding a new surface: when `diff_ref` is empty, the review prompt labels the diff as the uncommitted working tree plus staged area, so the model must not describe those findings as already committed or pushed.
- The same patch teaches the model Tlamatini's managed-secret convention: `agent/config.json` and selected `agent/agents/*/config.yaml` files can hold live local credentials in a keyed working copy, while `regen_secrets.py --mode push-able` scrubs them back to placeholders before commit.
- That guidance is mirrored into the `code-review` SKILL.md package too, keeping the chat-surface review behavior and the canvas Reviewer agent aligned on commit-state wording and secret-severity expectations.

Native-dialog spotlight

- The current tagged release `v1.4.2` removes Tkinter from the unstable runtime-facing dialog path and replaces it with `Tlamatini/agent/native_dialogs.py`, a native Windows dialog bridge used by browser-triggered pickers.
- This change pairs with the existing DB and operator dialogs: file and folder selection still feels local and GUI-first, but the fragile Tkinter dependency is no longer part of the interactive runtime path that users trigger from chat or ACP surfaces.
- The patch arrived with dedicated tests (`test_native_dialogs.py`) and with follow-up orphan-reaper/runtime adjustments, so the release reads as a stability pass rather than a cosmetic refactor.

Unrealer spotlight

- Unreal MCP is a UE5 plugin that runs inside the editor and listens on `127.0.0.1:55557` for one JSON command per TCP connection; Tlamatini is the client side of that link, not the plugin host.
- The Unrealer workflow agent forwards any command the connected plugin build exposes, up to a 53-command surface across nine categories: actor manipulation (incl. viewport screenshots), Blueprint authoring and node wiring, input mappings, UMG widget building, in-editor Python/console execution, level I/O, asset import, and material authoring. Headless build/cook/test is out of scope (it needs UnrealEditor-Cmd, not the editor socket).
- The same integration is available in both operator surfaces: checked Multi-Turn via `chat_agent_unrealer`, and ACP canvas flows via the visual Unrealer node plus Parametrizer chaining.
- Each Unrealer run is one command: load `config.yaml`, open a fresh TCP socket, send `{"type": command, "params": params}`, read until valid JSON arrives, and log one atomic `INI_SECTION_UNREALER<<< ... >>>END_SECTION_UNREALER` block.
- The wrapped-tool path stores chat runs under `agent/agents/pools/_chat_runs_/unrealer_<seq>_<id>/`, while visual flows use normal `unrealer_<n>` pool folders and can chain response fields through Parametrizer mappings.
- Exec Report treats Unrealer as its own row family, and troubleshooting stays concrete: connection-refused means the plugin is not listening, while read-timeout usually means UE5's game thread is busy.

De-Compresser spotlight

- De-Compresser is the deterministic archive worker for compression and decompression tasks, deciding direction from whichever side exposes a recognized archive extension.
- Supported archive families are `.gz`, `.zip`, `.7z`, `.tar.gz`, and `.gz.tar`; file-to-folder extraction and file-or-directory packing are both documented in the README and Book.
- Password handling is explicit: `passwordless=true` skips it, while `passwordless=false` requires the `DE_COMPRESSER_PWD` environment variable and fails fast if the secret is missing.
- The agent is reachable from both operator surfaces: ACP canvas nodes wire through dedicated connection-update views, and checked Multi-Turn can invoke it through `chat_agent_de_compresser`.

- Format engines are practical rather than magical: stdlib `gzip` and `zipfile` cover core cases, `7z` is preferred for encrypted `7z` or `zip`, and `py7zr` was added to `requirements.txt` as the Python fallback.
- Every run emits an `INI_SECTION_DE_COMPRESSER<<< ... >>>END_SECTION_DE_COMPRESSER` block and still triggers `target_agents`, so downstream Parametrizer or Raiser logic can branch on `success=true|false` instead of guessing from prose.

Orphan-reaper spotlight

- Tlamatini now ships a three-tier orphan reaper in `Tlamatini/agent/orphan_reaper.py` focused on Windows console-host leftovers such as `conhost.exe` and `openconsole.exe`.
- Tier 1 runs after spawn-capable Multi-Turn tool calls, Tier 2 runs once after the final answer in a background thread, and Tier 3 runs again during application shutdown through the same cleanup path that already tears down pools.
- The candidate set stays narrow on purpose: dead descendants of the current process tree, orphaned console hosts tied to that tree, and pool-linked processes whose `cmdline` still points into `agents/pools/...` even though tracking records are gone.

7. Repository Facts and Git Changes

Metric	Value
Repository inventory files	740
Tracked files in git	740
Git-unignored working-tree additions	0
Workflow agents	82
Multi-Turn tools	89
Wrapped chat-agent tools	57
Skills	27
agents_descriptions.md rows	82
Django migrations	144
Frontend JavaScript modules	30
Frontend CSS files	8
HTML templates	4
Python requirements	79
Binary/asset inventory files skipped from line count	26
Resolved version	1.26.0
Version source	git

Latest commits

Date	Commit	Subject
2026-06-19	d6a11b3	Fixing the ollama token stuff in docs.
2026-06-19	6903e12	Making clearer the installation/use steps.
2026-06-19	af7fdea	Adding some config images, and the getting start retargeted to whats most important.
2026-06-18	7c4de17	A little better redaction.
2026-06-18	33ff11e	version fix
2026-06-18	60c79a7	Keeping a short README.md
2026-06-17	8f6c725	Updating documentation in general to v1.26.0, by Tlamatini's-AutoBot using coworking with Claude+Codex.
2026-06-17	30a87f3	Updating documentation to the big dev yesterday and today (17/06/2026) Claude+Codex+Angela+Me(Tlamatini) implemented, about external mcp handling, by Tlamatini's-AutoBot.
2026-06-17	cb719bb	Re-check by Claude, Codex and angelahack.
2026-06-16	51d3ebd	feat(external-mcps): universal MCP client + MCP Doctor + full-surface Multi-Turn

Changes since the last committed PDF/PPTX refresh

Last committed visual-dossier refresh: 8f6c725 on 2026-06-17 — Updating documentation in general to v1.26.0, by Tlamatini's-AutoBot using coworking with Claude+Codex.

- The newest post-tag documentation wave is about usability rather than another runtime feature: BookOfTlamatini now begins with a five-step getting-started path, clarifies installer-vs-source choices, adds configuration screenshots, and explains that a localhost Ollama usually needs no token while a remote Ollama may.
- The live workspace now includes the untagged ESPHomer wave: a new ESPHome firmware agent, wrapped `chat_agent_esphomer` tool, sample YAML project, migrations, tests, and handbook chapters that extend Tlamatini into smart-home device provisioning.
- The markdown handbooks themselves were revised during that span, so this dossier refresh is carrying forward not only code/runtime changes but also the corrected operator-language and release-story wording.

Visual-dossier change appendix 1 of 1

Date	Commit	Subject
2026-06-19	d6a11b3	Fixing the ollama token stuff in docs.
2026-06-19	6903e12	Making clearer the installation/use steps.
2026-06-19	af7fdea	Adding some config images, and the getting start retargeted to whats most important.
2026-06-18	7c4de17	A little better redaction.
2026-06-18	33ff11e	version fix
2026-06-18	60c79a7	Keeping a short README.md

Git changes from the last 3 days

- The current Git window establishes the new `v1.26.0` baseline: the latest tag `v1.26.0` sits at commit `30a87f3` (June 17, 2026) and carries the External MCPs universal-client work — a config-driven client that connects to any MCP server over four transports (stdio / streamable-HTTP / SSE / WebSocket), the MCP Doctor diagnostic agent, full-surface Multi-Turn tool binding, and the Step-by-Step setup mode — on top of the earlier ESPHomer firmware lane.
- The latest handbook pass on June 19, 2026 shifts the emphasis from release bragging to operator onboarding: BookOfTlamatini now opens with an easy-follow five-step setup path (install Tlamatini, install/sign in to Ollama, pull models, configure the app, start using Multi-Turn only when needed), and it adds first-run configuration screenshots plus a clearer local-vs-remote Ollama-token rule.
- The tagged `v1.26.0` release now includes ESPHomer as a fourth firmware lane, bridging Tlamatini to ESPHome so she can author YAML device configs, validate, compile, upload, and observe smart-home firmware from chat or canvas.
- The latest documentation pass aligns the handbook and source with `v1.26.0`, which matters here because some older badges or prose lines still lag behind the live 82-agent / 89-tool inventory.
- Release-identity work also remains relevant: the SemVer policy, git-tag sourcing, runtime version surfaces, and build-time embedding across the Windows artefacts now define how Tlamatini reports her version.
- Multi-Turn behavior kept evolving across the period through quota tuning, execution-table persistence, autonomous-action improvements, and broader tool enablement.
- Documentation itself changed during the recent window, so the regenerated dossier is part of the tracked operator surface rather than an external afterthought.

3-day commit appendix 1 of 1

Date	Commit	Subject
2026-06-19	d6a11b3	Fixing the ollama token stuff in docs.
2026-06-19	6903e12	Making clearer the installation/use steps.
2026-06-19	af7fdea	Adding some config images, and the getting start retargeted to whats most important.
2026-06-18	7c4de17	A little better redaction.
2026-06-18	33ff11e	version fix
2026-06-18	60c79a7	Keeping a short README.md
2026-06-17	8f6c725	Updating documentation in general to v1.26.0, by Tlamatini's-AutoBot using coworking with Claude+Codex.
2026-06-17	30a87f3	Updating documentation to the big dev yesterday and today (17/06/2026) Claude+Codex+Angela+Me(Tlamatini) implemented, about external mcp handling, by Tlamatini's-AutoBot.
2026-06-17	cb719bb	Re-check by Claude, Codex and angelahack.
2026-06-16	51d3ebd	feat(external-mcps): universal MCP client + MCP Doctor + full-surface Multi-Turn

8. Effective Line Inventory by Language

Methodology: tracked text files only. Blank lines and comment-only lines are excluded. Python counts also remove module, class, and function docstrings detected through AST parsing.

Language	Files	Effective lines	Total lines	Share
Python	401	109,214	153,274	71.4%
Markdown	149	17,626	22,263	11.5%
JavaScript	30	15,047	18,769	9.8%
CSS	8	4,531	5,684	3.0%
YAML	88	1,777	4,486	1.2%
HTML	4	936	954	0.6%
Other text	7	912	1,108	0.6%
Prompt template	2	883	995	0.6%
PowerShell	6	672	987	0.4%
JSON	9	655	655	0.4%
JavaScript module	1	412	481	0.3%
Text	7	252	282	0.2%
Protocol Buffers	1	20	33	0.0%
Batch	1	15	29	0.0%

Total effective lines: 152,952

9. Largest Effective Source Files

Path	Language	Effective	Total
Tlamatini/agent/views.py	Python	8,132	11,824
Tlamatini/agent/tests.py	Python	5,318	6,889
Tlamatini/agent/tools.py	Python	2,772	4,050
Tlamatini/agent/doc_generation/complete_project_docs.py	Python	2,705	3,045
BookOfTlamatini.md	Markdown	2,400	3,405
Tlamatini/agent/agents/flowcreator/agentic_skill.md	Markdown	2,236	2,467
Tlamatini/agent/static/agent/css/agentic_control_panel.css	CSS	1,891	2,535
Tlamatini/agent/external_mcp_manager.py	Python	1,802	2,184
Tlamatini/agent/chat_agent_registry.py	Python	1,516	1,544
Tlamatini/agent/static/agent/css/agent_page.css	CSS	1,499	1,851
Tlamatini/agent/static/agent/js/acp-agent-connectors.js	JavaScript	1,469	1,629
Tlamatini/agent/static/agent/js/agent_page_chat.js	JavaScript	1,460	1,981
Tlamatini/agent/static/agent/js/agent_page_init.js	JavaScript	1,460	1,713
Tlamatini/agent/static/agent/js/acp-canvas-core.js	JavaScript	1,365	1,675
Tlamatini/agent/consumers.py	Python	1,303	1,641
Tlamatini/agent/agents/stm32er/stm32er.py	Python	1,212	1,684
Tlamatini/agent/test_stm32er_agent.py	Python	1,200	1,707
Tlamatini/agent/agents/crawler/crawler.py	Python	1,102	1,561
Tlamatini/agent/agents/whatstlamatini/whatstlamatini.py	Python	1,100	1,404
KIMI.md	Markdown	1,095	1,323
Tlamatini/agent/agents/arduiner/arduiner.py	Python	1,062	1,478
Tlamatini/agent/static/agent/js/acp-control-buttons.js	JavaScript	1,036	1,344
Tlamatini/agent/acpx/tests.py	Python	1,035	1,299
Tlamatini/agent/static/agent/js/agent_page_dialogs.js	JavaScript	998	1,220
Tlamatini/agent/mcp_agent.py	Python	987	1,611

10. Complete Repository File Tree (Repository Appendix)

Tree appendix 1 of 13

```
Tlamatini/
|-- .gitignore
|-- .mcp.json
|-- _acpx_pipeline_demo_report.md
|-- ACPX.md
|-- agents_descriptions.md
|-- apply_update.ps1
|-- BookOfTlamatini.md
|-- build.py
|-- build_installer.py
|-- build_uninstaller.py
|-- Claude-Desktop-KALI-MCP-Session.md
|-- CLAUDE.md
|-- copy_source_assets.py
|-- CreateShortcut.json
|-- CreateShortcut.ps1
|-- eslint.config.mjs
|-- example_of_prompt_to_make_de-compresser_agent.txt
|-- install.py
|-- KIMI.md
|-- LICENSE
|-- OllamaPricing.png
|-- package.json
|-- PIVOT_CHANGES.md
|-- pnpm-lock.yaml
|-- prompt_example_2_create_agent.txt
|-- README.md
|-- regen_secrets.py
|-- register_flw.ps1
|-- RemoveShortcut.ps1
|-- requirements.txt
|-- Tlamatini-Kali-Setup.md
|-- Tlamatini.ico
|-- Tlamatini.jpg
|-- Tlamatini.ps1
|-- tlamatini_app_summary.pdf
|-- Tlamatini_eXtended_Artificial_Intelligence_Humanly_Tempered.pptx
|-- TLAMATINI_MCP.md
|-- tlamatini_mcp_server.py
|-- TlamatiniJudgementDay.md
|-- uninstall.py
|-- unregister_flw.ps1
|-- VERSIONING.md
|-- versioning.py
|-- .claude/
|   |-- settings.json
|   |-- hooks/
|   |   |-- announce_skills.py
|   |   `-- test_announce_skills.py
|   |-- memory/
|   |   |-- feedback_agent_naming_conventions.md
|   |   |-- feedback_dont_overbuild_exec_safety.md
|   |   |-- feedback_hard_real_scenario_tests.md
|   |   |-- feedback_main_branch_only.md
|   |   |-- feedback_package_json_version_bump.md
|   |   |-- feedback_run_tlamatini_agents_visible.md
|   |   |-- feedback_state_constraints_upfront.md
|   |   |-- feedback_track_changes_pivot_file.md
|   |   |-- feedback_update_agent_docs.md
|   |   |-- feedback_user_owns_git.md
|   |   |-- MEMORY.md
|   |   |-- project_acpx_oneshot_prompt.md
|   |   |-- project_acpx_skills_menu.md
|   |   |-- project_acpx_toggle_skills.md
|   |   |-- project_acpxer_added.md
|   |   |-- project_arduiner_agent.md
|   |   |-- project_ask_execs_feature.md
|   |   |-- project_audioplayer_agent.md
|   |   |-- project_build_concurrency_guard.md
|   |   |-- project_build_tests_and_installer_enterkey.md
|   |   |-- project_camcorder_agent.md
|   |   |-- project_carried_python_for_agents.md
|   |   |-- project_conjunction_parser_fix.md
|   |   |-- project_daily_chat_test_skill.md
|   |   |-- project_desktop_ui_lifecycle.md
|   |   |-- project_director_virtuoso_demo_prompts.md
```

Tree appendix 2 of 13

```
|   |   |-- project_doc_refresh_2026_05_06.md
|   |   |-- project_doc_refresh_2026_05_08.md
|   |   |-- project_doc_refresh_2026_05_09.md
|   |   |-- project_eight_fixes_2026_05_07.md
|   |   |-- project_emailer_recmailer_oneshot.md
|   |   |-- project_embedding_memory_guard.md
|   |   |-- project_esp32er_agent.md
|   |   |-- project_execute_command_bounded_fix.md
|   |   |-- project_execute_file_foreground_fix.md
|   |   |-- project_external_exec_safety_layer.md
|   |   |-- project_flow_compiler_dialog_wins.md
|   |   |-- project_kalier_agent.md
|   |   |-- project_keyboarder_wrapped_2026_05_07.md
|   |   |-- project_living_canvas_dropped.md
|   |   |-- project_mouser_wrapped_2026_05_07.md
|   |   |-- project_native_picker_tcltk_fix.md
|   |   |-- project_native_toast.md
|   |   |-- project_numpy_pyinstaller.md
|   |   |-- project_ollama_source_build_breaks_embeddings.md
|   |   |-- project_planner_followup.md
|   |   |-- project_playwrighter_agent.md
|   |   |-- project_playwrighter_hold_open.md
|   |   |-- project_prompt_catalog_mode_badges.md
|   |   |-- project_pythonxer_forkbomb_fix.md
|   |   |-- project_pythonxer_strict_ruff_gate.md
|   |   |-- project_quote_apostrophe_fix_2026_05_07.md
|   |   |-- project_reaper_console_window_fix.md
|   |   |-- project_recorder_agent.md
|   |   |-- project_reviewer_analyzer_agents.md
|   |   |-- project_reviewer_analyzer_demo_prompts.md
|   |   |-- project_reviewer_committed_secrets_falsepos.md
|   |   |-- project_secret_leak_recovery.md
|   |   |-- project_stm32er_agent.md
|   |   |-- project_stm32er_bootstrap_preflight.md
|   |   |-- project_stm32er_demo_prompts.md
|   |   |-- project_stm32er_docs_libs_parametrizer.md
|   |   |-- project_stm32er_hil_serial_proof_fix.md
|   |   |-- project_stm32er_tests.md
|   |   |-- project_summarizer_oneshot_mode.md
|   |   |-- project_taskbar_flash_attention.md
|   |   |-- project_teletlamatini_acpx.md
|   |   |-- project_temp_templates_policy.md
|   |   |-- project_unity_mcp_declined.md
|   |   |-- project_unrealer_expanded_surface.md
|   |   |-- project_versioning_2026_05_15.md
|   |   |-- project_videoplayer_agent.md
|   |   |-- project_windower_agent.md
|   |   `-- user_profile.md
|   `-- skills/
|       |-- tlamatini-agent-creation/
|       |   `-- SKILL.md
|       |-- tlamatini-agent-naming/
|       |   `-- SKILL.md
|       |-- tlamatini-daily-chat-test/
|       |   |-- .gitignore
|       |   |-- SKILL.md
|       |   `-- harness/
|       |       |-- config.py
|       |       |-- cyber_watchdog_test.py
|       |       |-- mcp_playwright_suite.py
|       |       |-- qualify.py
|       |       |-- questions.py
|       |       |-- README.md
|       |       |-- requirements.txt
|       |       |-- run_test.py
|       |       |-- talk_test.py
|       |       `-- wrapped_questions.py
|       |-- tlamatini-self-modify-inclusion/
|       |   |-- SKILL.md
|       |   `-- scripts/
|       |       `-- sweep_self_modify.py
|       `-- tlamatini-self-update-inclusion/
|           |-- SKILL.md
|           `-- scripts/
|               `-- sweep_self_update.py
|-- .codex/
```

Tree appendix 3 of 13

```
|   `-- skills/
|       |-- full-project-pdf-dossier/
|       |   `-- SKILL.md
|       `-- overlap-safe-pptx-dossier/
|           `-- SKILL.md
|-- .github/
|   `-- workflows/
|       `-- name-guard.yml
|-- .scanning/
|   `-- finding-policy.json
|-- AuxTests/
|   `-- mypdf2.py
|-- docs/
|   |-- external_mcp_bulletproof_architecture.md
|   `-- claude/
|       |-- acpx.md
|       |-- agents.md
|       |-- architecture.md
|       |-- exec-report.md
|       |-- frontend.md
|       |-- gotchas.md
|       |-- INDEX.md
|       |-- mcp-tools.md
|       |-- multi-turn.md
|       `-- recent-fixes.md
|-- pyinstaller_hooks/
|   `-- hook-numpy.py
`-- Tlamatini/
    |-- cat_art.py
    |-- manage.py
    |-- .agents/
    |   `-- workflows/
    |       `-- create_new_agent.md
    |-- .mcps/
    |   `-- create_new_mcp.md
    |-- .skills/
    |   `-- create_new_skill.md
    |-- agent/
    |   |-- __init__.py
    |   |-- access_key_wizard.py
    |   |-- admin.py
    |   |-- apps.py
    |   |-- capability_registry.py
    |   |-- chain_files_search_lcel.py
    |   |-- chain_system_lcel.py
    |   |-- chat_agent_registry.py
    |   |-- chat_agent_runtime.py
    |   |-- chat_history_loader.py
    |   |-- command_togen_pb2.txt
    |   |-- command_watchdog.py
    |   |-- config.json
    |   |-- config_loader.py
    |   |-- constants.py
    |   |-- consumers.py
    |   |-- embedding_memory_guard.py
    |   |-- exec_permission.py
    |   |-- external_mcp_manager.py
    |   |-- external_mcps.json
    |   |-- filesearch.proto
    |   |-- filesearch_pb2.py
    |   |-- filesearch_pb2_grpc.py
    |   |-- global_execution_planner.py
    |   |-- global_state.py
    |   |-- gpu_perf.py
    |   |-- inet_determiner.py
    |   |-- mcp_agent.py
    |   |-- mcp_files_search_client.py
    |   |-- mcp_files_search_server.py
    |   |-- mcp_system_client.py
    |   |-- mcp_system_server.py
    |   |-- models.py
    |   |-- native_dialogs.py
    |   |-- orphan_reaper.py
    |   |-- path_guard.py
    |   |-- prompt.pmt
    |   |-- rag_enhancements.py
```

Tree appendix 4 of 13

```
|   |-- routing.py
|   |-- self_update.py
|   |-- test_access_key_wizard.py
|   |-- test_arduiner_agent.py
|   |-- test_audioplayer_agent.py
|   |-- test_blenderer_agent.py
|   |-- test_build_scripts.py
|   |-- test_camcorder_agent.py
|   |-- test_chat_bridge_bots.py
|   |-- test_command_watchdog.py
|   |-- test_crawler_agent.py
|   |-- test_de_compresser.py
|   |-- test_editor_agent.py
|   |-- test_embedding_memory_guard.py
|   |-- test_esp32er_agent.py
|   |-- test_esphomer_agent.py
|   |-- test_execute_command_bounded.py
|   |-- test_external_mcp_add_flow.py
|   |-- test_external_mcp_e2e.py
|   |-- test_external_mcp_transports.py
|   |-- test_external_mcp_universal.py
|   |-- test_file_extractor_reader.py
|   |-- test_flow_contracts.py
|   |-- test_globber_agent.py
|   |-- test_googler_agent.py
|   |-- test_grepper_agent.py
|   |-- test_j_decompiler_agent.py
|   |-- test_kalier_agent.py
|   |-- test_native_dialogs.py
|   |-- test_orphan_reaper.py
|   |-- test_parametrizer_mcp_doctor.py
|   |-- test_password_quoting.py
|   |-- test_playwrighter_agent.py
|   |-- test_recorder_agent.py
|   |-- test_self_update.py
|   |-- test_step_by_step_mode.py
|   |-- test_stm32er_agent.py
|   |-- test_talker_agent.py
|   |-- test_temp_dir_policy.py
|   |-- test_unrealer_agent.py
|   |-- test_videoplayer_agent.py
|   |-- test_whisperer_agent.py
|   |-- test_windower_agent.py
|   |-- test_windows_app_registration.py
|   |-- tests.py
|   |-- Tlamatini.md
|   |-- tools.py
|   |-- urls.py
|   |-- version.py
|   |-- views.py
|   |-- web_search_llm.py
|   |-- window_flash.py
|   |-- windows_app_registration.py
|   |-- acpx/
|   |   |-- __init__.py
|   |   |-- agent_registry.py
|   |   |-- config.py
|   |   |-- permissions.py
|   |   |-- runtime.py
|   |   |-- service.py
|   |   |-- session_store.py
|   |   |-- tests.py
|   |   |-- tools.py
|   |   `-- windows_spawn.py
|   |-- agents/
|   |   |-- acpxer/
|   |   |   |-- acpxer.py
|   |   |   `-- config.yaml
|   |   |-- analyzer/
|   |   |   |-- analyzer.py
|   |   |   `-- config.yaml
|   |   |-- and/
|   |   |   |-- and.py
|   |   |   `-- config.yaml
|   |   |-- apirer/
|   |   |   |-- apirer.py
```

Tree appendix 5 of 13

```
|   |   |   `-- config.yaml
|   |   |-- arduiner/
|   |   |   |-- arduiner.py
|   |   |   |-- config.yaml
|   |   |   `-- ArduinoTemplateProject/
|   |   |       |-- ArduinoTemplateProject.ino
|   |   |       |-- README.md
|   |   |       |-- sketch.yaml
|   |   |       `-- src/
|   |   |           |-- Heartbeat.cpp
|   |   |           `-- Heartbeat.h
|   |   |-- asker/
|   |   |   |-- asker.py
|   |   |   `-- config.yaml
|   |   |-- audioplayer/
|   |   |   |-- audioplayer.py
|   |   |   `-- config.yaml
|   |   |-- barrier/
|   |   |   |-- barrier.py
|   |   |   |-- config.yaml
|   |   |   `-- test_barrier.py
|   |   |-- blenderer/
|   |   |   |-- blenderer.py
|   |   |   `-- config.yaml
|   |   |-- camcorder/
|   |   |   |-- camcorder.py
|   |   |   `-- config.yaml
|   |   |-- cleaner/
|   |   |   |-- __init__.py
|   |   |   |-- cleaner.py
|   |   |   `-- config.yaml
|   |   |-- counter/
|   |   |   |-- config.yaml
|   |   |   `-- counter.py
|   |   |-- crawler/
|   |   |   |-- config.yaml
|   |   |   `-- crawler.py
|   |   |-- croner/
|   |   |   |-- config.yaml
|   |   |   `-- croner.py
|   |   |-- de_compresser/
|   |   |   |-- config.yaml
|   |   |   `-- de_compresser.py
|   |   |-- deleter/
|   |   |   |-- config.yaml
|   |   |   `-- deleter.py
|   |   |-- dockerer/
|   |   |   |-- config.yaml
|   |   |   `-- dockerer.py
|   |   |-- editor/
|   |   |   |-- config.yaml
|   |   |   `-- editor.py
|   |   |-- emailer/
|   |   |   |-- config.yaml
|   |   |   `-- emailer.py
|   |   |-- ender/
|   |   |   |-- config.yaml
|   |   |   `-- ender.py
|   |   |-- esp32er/
|   |   |   |-- config.yaml
|   |   |   `-- esp32er.py
|   |   |-- esphomer/
|   |   |   |-- config.yaml
|   |   |   |-- esphomer.py
|   |   |   `-- ESPHomeTemplateProject/
|   |   |       |-- README.md
|   |   |       `-- tlamatini-light.yaml
|   |   |-- executer/
|   |   |   |-- config.yaml
|   |   |   `-- executer.py
|   |   |-- file_creator/
|   |   |   |-- config.yaml
|   |   |   `-- file_creator.py
|   |   |-- file_extractor/
|   |   |   |-- config.yaml
|   |   |   `-- file_extractor.py
```

Tree appendix 6 of 13

```
|   |   |-- file_interpreter/
|   |   |   |-- config.yaml
|   |   |   `-- file_interpreter.py
|   |   |-- flowbacker/
|   |   |   |-- config.yaml
|   |   |   `-- flowbacker.py
|   |   |-- flowcreator/
|   |   |   |-- agentic_skill.md
|   |   |   |-- config.yaml
|   |   |   `-- flowcreator.py
|   |   |-- flowhypervisor/
|   |   |   |-- config.yaml
|   |   |   |-- flowhypervisor.py
|   |   |   |-- hypervisor_alert.wav
|   |   |   `-- monitoring-prompt.pmt
|   |   |-- forker/
|   |   |   |-- config.yaml
|   |   |   `-- forker.py
|   |   |-- gateway_relayer/
|   |   |   |-- config.yaml
|   |   |   |-- gateway_relayer.md
|   |   |   `-- gateway_relayer.py
|   |   |-- gatewayer/
|   |   |   |-- config.yaml
|   |   |   |-- gatewayer.md
|   |   |   |-- gatewayer.py
|   |   |   `-- gatewayer_explanation.md
|   |   |-- gitter/
|   |   |   |-- config.yaml
|   |   |   `-- gitter.py
|   |   |-- globber/
|   |   |   |-- config.yaml
|   |   |   `-- globber.py
|   |   |-- googler/
|   |   |   |-- config.yaml
|   |   |   `-- googler.py
|   |   |-- grepper/
|   |   |   |-- config.yaml
|   |   |   `-- grepper.py
|   |   |-- image_interpreter/
|   |   |   |-- config.yaml
|   |   |   `-- image_interpreter.py
|   |   |-- j_decompiler/
|   |   |   |-- config.yaml
|   |   |   `-- j_decompiler.py
|   |   |-- jenkinser/
|   |   |   |-- config.yaml
|   |   |   `-- jenkinser.py
|   |   |-- kalier/
|   |   |   |-- config.yaml
|   |   |   `-- kalier.py
|   |   |-- keyboarder/
|   |   |   |-- config.yaml
|   |   |   `-- keyboarder.py
|   |   |-- kuberneter/
|   |   |   |-- config.yaml
|   |   |   `-- kuberneter.py
|   |   |-- kyber_cipher/
|   |   |   |-- config.yaml
|   |   |   `-- kyber_cipher.py
|   |   |-- kyber_decipher/
|   |   |   |-- config.yaml
|   |   |   `-- kyber_decipher.py
|   |   |-- kyber_keygen/
|   |   |   |-- config.yaml
|   |   |   `-- kyber_keygen.py
|   |   |-- mcp_doctor/
|   |   |   |-- config.yaml
|   |   |   `-- mcp_doctor.py
|   |   |-- mongoxer/
|   |   |   |-- config.yaml
|   |   |   `-- mongoxer.py
|   |   |-- monitor_log/
|   |   |   |-- config.yaml
|   |   |   `-- monitor_log.py
|   |   |-- monitor_netstat/
```

Tree appendix 7 of 13

```
|   |   |   |-- config.yaml
|   |   |   `-- monitor_netstat.py
|   |   |-- mouser/
|   |   |   |-- config.yaml
|   |   |   `-- mouser.py
|   |   |-- mover/
|   |   |   |-- config.yaml
|   |   |   `-- mover.py
|   |   |-- node_manager/
|   |   |   |-- config.yaml
|   |   |   |-- node_manager.md
|   |   |   |-- node_manager.py
|   |   |   |-- nodes_example.json
|   |   |   `-- nodes_example.yaml
|   |   |-- notifier/
|   |   |   |-- config.yaml
|   |   |   `-- notifier.py
|   |   |-- or/
|   |   |   |-- config.yaml
|   |   |   `-- or.py
|   |   |-- parametrizer/
|   |   |   |-- config.yaml
|   |   |   `-- parametrizer.py
|   |   |-- playwrighter/
|   |   |   |-- config.yaml
|   |   |   `-- playwrighter.py
|   |   |-- prompter/
|   |   |   |-- config.yaml
|   |   |   `-- prompter.py
|   |   |-- pser/
|   |   |   |-- config.yaml
|   |   |   `-- pser.py
|   |   |-- pythonxer/
|   |   |   |-- config.yaml
|   |   |   `-- pythonxer.py
|   |   |-- raiser/
|   |   |   |-- config.yaml
|   |   |   `-- raiser.py
|   |   |-- recmailer/
|   |   |   |-- config.yaml
|   |   |   `-- recmailer.py
|   |   |-- recorder/
|   |   |   |-- config.yaml
|   |   |   `-- recorder.py
|   |   |-- reviewer/
|   |   |   |-- config.yaml
|   |   |   `-- reviewer.py
|   |   |-- scper/
|   |   |   |-- config.yaml
|   |   |   `-- scper.py
|   |   |-- shoter/
|   |   |   |-- config.yaml
|   |   |   `-- shoter.py
|   |   |-- sleeper/
|   |   |   |-- config.yaml
|   |   |   `-- sleeper.py
|   |   |-- sqler/
|   |   |   |-- config.yaml
|   |   |   `-- sqler.py
|   |   |-- ssher/
|   |   |   |-- config.yaml
|   |   |   `-- ssher.py
|   |   |-- starter/
|   |   |   |-- config.yaml
|   |   |   `-- starter.py
|   |   |-- stm32er/
|   |   |   |-- config.yaml
|   |   |   `-- stm32er.py
|   |   |-- stopper/
|   |   |   |-- config.yaml
|   |   |   `-- stopper.py
|   |   |-- summarizer/
|   |   |   |-- config.yaml
|   |   |   `-- summarizer.py
|   |   |-- talker/
|   |   |   |-- config.yaml
```

Tree appendix 8 of 13

```
|   |   |   `-- talker.py
|   |   |-- telegramer/
|   |   |   |-- config.yaml
|   |   |   `-- telegramer.py
|   |   |-- telegramrx/
|   |   |   |-- config.yaml
|   |   |   `-- telegramrx.py
|   |   |-- teletlamatini/
|   |   |   |-- config.yaml
|   |   |   `-- teletlamatini.py
|   |   |-- unrealer/
|   |   |   |-- config.yaml
|   |   |   `-- unrealer.py
|   |   |-- videoplayer/
|   |   |   |-- config.yaml
|   |   |   `-- videoplayer.py
|   |   |-- whatsapper/
|   |   |   |-- config.yaml
|   |   |   `-- whatsapper.py
|   |   |-- whatstlamatini/
|   |   |   |-- config.yaml
|   |   |   `-- whatstlamatini.py
|   |   |-- whisperer/
|   |   |   |-- config.yaml
|   |   |   `-- whisperer.py
|   |   `-- windower/
|   |       |-- config.yaml
|   |       `-- windower.py
|   |-- doc_generation/
|   |   |-- complete_project_docs.py
|   |   |-- mardown_to_pdf.py
|   |   |-- refresh_project_docs.py
|   |   `-- test_output.pdf
|   |-- images/
|   |   |-- ACPXKeysConfigureWizard.jpg
|   |   |-- ConfigureModels.jpg
|   |   |-- HiIamTlamatini.png
|   |   |-- MenuConfig.jpg
|   |   |-- mockup.jpg
|   |   |-- RealisticTlamatini.png
|   |   |-- ShouldersUoTlamatini.png
|   |   |-- Tlamatini AI Agentic Advisor Video.mp4
|   |   |-- TlamatiniAbout.png
|   |   |-- TlamatiniAndKyber.mp4
|   |   |-- TlamatiniDrawings.png
|   |   |-- TlammatiniLetsMakeSomeMagic.mp4
|   |   |-- TorsoTlamatini.png
|   |   `-- XAIHT-Tlamatini.mp4
|   |-- imaging/
|   |   |-- converter.py
|   |   |-- image_interpreter.py
|   |   `-- screenshot.py
|   |-- management/
|   |   |-- __init__.py
|   |   `-- commands/
|   |       |-- acpx_lint.py
|   |       `-- startserver.py
|   |-- migrations/
|   |   |-- 0001_initial.py
|   |   |-- 0002_populate_db.py
|   |   |-- 0003_add_cleaner.py
|   |   |-- 0004_add_deleter.py
|   |   |-- 0005_add_executer.py
|   |   |-- 0006_add_notifier.py
|   |   |-- 0007_add_stopper.py
|   |   |-- 0008_add_recmailer.py
|   |   |-- 0009_add_whatsapper.py
|   |   |-- 0010_add_pythonxer.py
|   |   |-- 0011_add_asker.py
|   |   |-- 0012_repopulate_all_agents.py
|   |   |-- 0013_add_forker.py
|   |   |-- 0014_repopulate_all_agents.py
|   |   |-- 0015_add_shoter.py
|   |   |-- 0016_repopulate_all_agents.py
|   |   |-- 0017_add_ssher.py
|   |   |-- 0018_repopulate_all_agents.py
```

Tree appendix 9 of 13

```
|   |   |-- 0019_add_scper.py
|   |   |-- 0020_repopulate_all_agents.py
|   |   |-- 0021_add_telegramrx.py
|   |   |-- 0022_repopulate_all_agents.py
|   |   |-- 0023_add_mongoxer.py
|   |   |-- 0024_repopulate_all_agents.py
|   |   |-- 0025_add_telegramer.py
|   |   |-- 0026_repopulate_all_agents.py
|   |   |-- 0027_add_sqler.py
|   |   |-- 0028_repopulate_all_agents.py
|   |   |-- 0029_add_prompter.py
|   |   |-- 0030_repopulate_all_agents.py
|   |   |-- 0031_add_flowcreator.py
|   |   |-- 0032_repopulate_all_agents.py
|   |   |-- 0033_add_gitter.py
|   |   |-- 0034_add_dockerer.py
|   |   |-- 0035_add_pser.py
|   |   |-- 0036_add_kuberneter.py
|   |   |-- 0037_add_apirer.py
|   |   |-- 0038_add_jenkinser.py
|   |   |-- 0039_add_crawler.py
|   |   |-- 0040_add_summarizer.py
|   |   |-- 0041_add_flowhypervisor.py
|   |   |-- 0042_add_mouser.py
|   |   |-- 0043_agentmessage_conversation_user.py
|   |   |-- 0044_add_counter.py
|   |   |-- 0045_add_file_interpreter.py
|   |   |-- 0046_add_image_interpreter.py
|   |   |-- 0047_add_gatewayer.py
|   |   |-- 0048_add_gateway_relayer.py
|   |   |-- 0049_add_node_manager.py
|   |   |-- 0050_add_file_creator.py
|   |   |-- 0051_add_file_extractor.py
|   |   |-- 0052_add_kyber_keygen.py
|   |   |-- 0053_add_kyber_cipher.py
|   |   |-- 0054_add_kyber_decipher.py
|   |   |-- 0055_fix_kyber_cipher_names.py
|   |   |-- 0056_add_parametrizer.py
|   |   |-- 0057_add_flowbacker.py
|   |   |-- 0058_add_barrier.py
|   |   |-- 0059_add_j_decompiler.py
|   |   |-- 0060_add_agent_parametrizer_tool.py
|   |   |-- 0061_add_agent_starter_tool.py
|   |   |-- 0062_sync_agent_control_tools_and_prompts.py
|   |   |-- 0063_add_agent_parametrizer_prompt.py
|   |   |-- 0064_add_chat_agent_run_model.py
|   |   |-- 0065_add_chat_wrapped_agent_tools.py
|   |   |-- 0066_add_keyboarder.py
|   |   |-- 0067_add_multi_turn_demo_prompts.py
|   |   |-- 0068_add_googler_tool.py
|   |   |-- 0069_add_googler_agent.py
|   |   |-- 0070_add_teletlamatini.py
|   |   |-- 0071_acpx_skills.py
|   |   |-- 0072_add_acpx_demo_prompts.py
|   |   |-- 0073_acpx_demo_gemini_uplift.py
|   |   |-- 0074_simplify_demo_prompts.py
|   |   |-- 0075_add_acpx_tool_surface.py
|   |   |-- 0076_add_acpxer.py
|   |   |-- 0077_add_whatstlamatini.py
|   |   |-- 0078_add_chat_agent_keyboarder_tool.py
|   |   |-- 0079_add_chat_agent_mouser_tool.py
|   |   |-- 0080_add_chat_agent_sleeper_tool.py
|   |   |-- 0081_add_window_present_and_run_wait_tools.py
|   |   |-- 0082_add_chat_agent_j_decompiler_tool.py
|   |   |-- 0083_add_de_compresser.py
|   |   |-- 0084_add_chat_agent_de_compresser_tool.py
|   |   |-- 0085_add_unrealer.py
|   |   |-- 0086_add_chat_agent_unrealer_tool.py
|   |   |-- 0087_add_unrealer_demo_prompt.py
|   |   |-- 0088_add_reviewer.py
|   |   |-- 0089_add_analyzer.py
|   |   |-- 0090_add_reviewer_analyzer_demo_prompts.py
|   |   |-- 0091_add_playwrighter.py
|   |   |-- 0092_add_chat_agent_playwrighter_tool.py
|   |   |-- 0093_add_windower.py
|   |   |-- 0094_add_chat_agent_windower_tool.py
```

Tree appendix 10 of 13

```
|   |   |-- 0095_add_windower_playwrighter_demo_prompts.py
|   |   |-- 0096_add_director_virtuoso_demo_prompts.py
|   |   |-- 0097_add_kalier.py
|   |   |-- 0098_add_chat_agent_kalier_tool.py
|   |   |-- 0099_add_kalier_demo_prompts.py
|   |   |-- 0100_add_unrealer_extended_demo_prompts.py
|   |   |-- 0101_add_stm32er.py
|   |   |-- 0102_add_chat_agent_stm32er_tool.py
|   |   |-- 0103_add_stm32er_demo_prompts.py
|   |   |-- 0104_fix_stm32er_hil_serial_proof.py
|   |   |-- 0105_add_esp32er.py
|   |   |-- 0106_add_chat_agent_esp32er_tool.py
|   |   |-- 0107_add_esp32er_demo_prompts.py
|   |   |-- 0108_add_flow_making_demo_prompt.py
|   |   |-- 0109_add_arduiner.py
|   |   |-- 0110_add_chat_agent_arduiner_tool.py
|   |   |-- 0111_add_arduiner_demo_prompts.py
|   |   |-- 0112_add_camcorder.py
|   |   |-- 0113_add_chat_agent_camcorder_tool.py
|   |   |-- 0114_add_recorder.py
|   |   |-- 0115_add_chat_agent_recorder_tool.py
|   |   |-- 0116_add_audioplayer.py
|   |   |-- 0117_add_chat_agent_audioplayer_tool.py
|   |   |-- 0118_add_videoplayer.py
|   |   |-- 0119_add_chat_agent_videoplayer_tool.py
|   |   |-- 0120_add_talker.py
|   |   |-- 0121_add_chat_agent_talker_tool.py
|   |   |-- 0122_add_talker_demo_prompt.py
|   |   |-- 0123_add_whisperer.py
|   |   |-- 0124_add_chat_agent_whisperer_tool.py
|   |   |-- 0125_add_whisperer_demo_prompt.py
|   |   |-- 0126_add_blenderer.py
|   |   |-- 0127_add_chat_agent_blenderer_tool.py
|   |   |-- 0128_add_blenderer_demo_prompts.py
|   |   |-- 0129_add_editor.py
|   |   |-- 0130_add_chat_agent_editor_tool.py
|   |   |-- 0131_add_editor_demo_prompts.py
|   |   |-- 0132_add_grepper.py
|   |   |-- 0133_add_chat_agent_grepper_tool.py
|   |   |-- 0134_add_grepper_demo_prompts.py
|   |   |-- 0135_add_globber.py
|   |   |-- 0136_add_chat_agent_globber_tool.py
|   |   |-- 0137_add_globber_demo_prompts.py
|   |   |-- 0138_add_esphomer.py
|   |   |-- 0139_add_chat_agent_esphomer_tool.py
|   |   |-- 0140_add_esphomer_demo_prompts.py
|   |   |-- 0141_add_mcp_doctor.py
|   |   |-- 0142_add_chat_agent_mcp_doctor_tool.py
|   |   |-- 0143_add_mcp_doctor_demo_prompt.py
|   |   |-- 0144_insert_java17_maven_demo_prompt_at_5.py
|   |   `-- __init__.py
|   |-- monitors/
|   |   `-- monitor_sql.py
|   |-- opus_client/
|   |   |-- claude_opus_client.py
|   |   |-- examples.py
|   |   `-- README.md
|   |-- rag/
|   |   |-- __init__.py
|   |   |-- config.py
|   |   |-- factory.py
|   |   |-- interaction.py
|   |   |-- interface.py
|   |   |-- loaders.py
|   |   |-- prompts.py
|   |   |-- retrieval.py
|   |   |-- splitters.py
|   |   |-- utils.py
|   |   `-- chains/
|   |       |-- base.py
|   |       |-- basic.py
|   |       |-- history_aware.py
|   |       `-- unified.py
|   |-- services/
|   |   |-- __init__.py
|   |   |-- agent_contracts.py
```

Tree appendix 11 of 13

```
|   |   |-- agent_paths.py
|   |   |-- answer_analizer.py
|   |   |-- filesystem.py
|   |   |-- flow_compiler.py
|   |   |-- flow_spec.py
|   |   `-- response_parser.py
|   |-- skills/
|   |   |-- __init__.py
|   |   |-- frontmatter.py
|   |   |-- harness.py
|   |   |-- io_contract.py
|   |   `-- registry.py
|   |-- skills_pkg/
|   |   |-- _meta/
|   |   |   |-- lint.py
|   |   |   `-- schema.json
|   |   |-- acp_router/
|   |   |   `-- SKILL.md
|   |   |-- code_review/
|   |   |   `-- SKILL.md
|   |   |-- create_new_agent/
|   |   |   `-- SKILL.md
|   |   |-- create_new_mcp/
|   |   |   `-- SKILL.md
|   |   |-- flow_making/
|   |   |   |-- SKILL.md
|   |   |   |-- references/
|   |   |   |   `-- flw_schema.md
|   |   |   `-- scripts/
|   |   |       |-- make_flow.py
|   |   |       `-- result_to_flw.py
|   |   |-- github/
|   |   |   `-- SKILL.md
|   |   |-- gmail/
|   |   |   `-- SKILL.md
|   |   |-- hello_world/
|   |   |   `-- SKILL.md
|   |   |-- jira/
|   |   |   `-- SKILL.md
|   |   |-- kali_pentest/
|   |   |   `-- SKILL.md
|   |   |-- notion/
|   |   |   `-- SKILL.md
|   |   |-- security_audit/
|   |   |   `-- SKILL.md
|   |   |-- setup_new_acpx_key/
|   |   |   `-- SKILL.md
|   |   |-- skill_creator/
|   |   |   |-- SKILL.md
|   |   |   `-- scripts/
|   |   |       `-- quick_validate.py
|   |   |-- slack/
|   |   |   `-- SKILL.md
|   |   |-- summarize/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_allowed_hosts_tighten/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_csrf_exempt_audit/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_exec_report_row_adder/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_flow_from_objective/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_flw_doctor/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_new_acp_agent/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_planner_trace_replay/
|   |   |   `-- SKILL.md
|   |   |-- tlamatini_static_version_bumper/
|   |   |   `-- SKILL.md
|   |   |-- todoist/
|   |   |   `-- SKILL.md
|   |   |-- trello/
|   |   |   `-- SKILL.md
|   |   `-- weather/
```

Tree appendix 12 of 13

```
|   |       `-- SKILL.md
|   |-- static/
|   |   `-- agent/
|   |       |-- css/
|   |       |   |-- access_keys_wizard.css
|   |       |   |-- agent_page.css
|   |       |   |-- agentic_control_panel.css
|   |       |   |-- external_mcps_dialog.css
|   |       |   |-- login.css
|   |       |   |-- skills_dialog.css
|   |       |   |-- tools_dialog.css
|   |       |   `-- welcome.css
|   |       |-- img/
|   |       |   |-- spinner.svg
|   |       |   `-- Tlamatini.ico
|   |       |-- js/
|   |       |   |-- access_keys_wizard.js
|   |       |   |-- acp-agent-connectors.js
|   |       |   |-- acp-canvas-core.js
|   |       |   |-- acp-canvas-undo.js
|   |       |   |-- acp-control-buttons.js
|   |       |   |-- acp-file-io.js
|   |       |   |-- acp-flow-snapshot.js
|   |       |   |-- acp-globals.js
|   |       |   |-- acp-layout.js
|   |       |   |-- acp-parametrizer-dialog.js
|   |       |   |-- acp-running-state.js
|   |       |   |-- acp-session.js
|   |       |   |-- acp-undo-manager.js
|   |       |   |-- acp-validate.js
|   |       |   |-- agent_page_canvas.js
|   |       |   |-- agent_page_chat.js
|   |       |   |-- agent_page_context.js
|   |       |   |-- agent_page_dialogs.js
|   |       |   |-- agent_page_init.js
|   |       |   |-- agent_page_layout.js
|   |       |   |-- agent_page_state.js
|   |       |   |-- agent_page_ui.js
|   |       |   |-- agentic_control_panel.js
|   |       |   |-- canvas_item_dialog.js
|   |       |   |-- chat_page_runtime_poller.js
|   |       |   |-- contextual_menus.js
|   |       |   |-- external_mcps_dialog.js
|   |       |   |-- shared-runtime-dialogs.js
|   |       |   |-- skills_dialog.js
|   |       |   `-- tools_dialog.js
|   |       |-- sounds/
|   |       |   |-- hypervisor_alert.wav
|   |       |   `-- notification.wav
|   |       `-- video/
|   |           `-- XAIHT-Tlamatini.mp4
|   |-- telegram/
|   |   |-- AztecTlamatini.txt
|   |   |-- config.yaml
|   |   `-- telegram_receiver.py
|   |-- templates/
|   |   `-- agent/
|   |       |-- agent_page.html
|   |       |-- agentic_control_panel.html
|   |       |-- login.html
|   |       `-- welcome.html
|   |-- test_fixtures/
|   |   `-- talker_orpheus_tokens.txt
|   `-- TlamatiniSourceCode/
|       `-- README.md
|-- DB/
|   |-- Older/
|   |   `-- README.md
|   `-- ToLoad/
|       `-- README.md
|-- jd-cli/
|   |-- jd-cli.bat
|   `-- jd-cli.jar
`-- tlamatini/
    |-- __init__.py
    |-- asgi.py
```

Tree appendix 13 of 13

```
|-- context_processors.py
|-- logging_filters.py
|-- middleware.py
|-- settings.py
|-- urls.py
`-- wsgi.py
```